Отчёта по лабораторной работе

Иммитационное моделирование

Нджову Нелиа

Содержание

Список иллюстраций

Список таблиц

1 Цель работы

Целью данной лабораторной работы является практическое изучение моделей SIR и Лотки-Вольтерры с использованием языка Джулии.

2 Задание

1. Изучить модель SIR с помощью julia

2. Изучить модель Лотки-Вольтерры с помощью julia

3 Теоретическое введение

Модель SIR есть классическая и фундаментальная математическая модель эпидемиологии, описывающая распространение инфекционного заболевания в закрытой популяции. Другими словами, модель SIR является одной из самых базовых моделей для описания временной динамики инфекционного заболевания в популяции

Модель SIR делит всю популяцию на три взаимосвязанные группы (компартменты), что отражено в её названии:

— S — Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться.

— I — Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию.

— R — Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи.

Модель Лотки-Вольтерры — это фундаментальная математическая модель в экологии, описывающая динамику взаимодействия двух видов: хищников и жертв. Она была независимо предложена в 1920-х годах:

— Альфредом Лоткой (1925) для химических реакций

— Витторио Вольтеррой (1926) для объяснения колебаний улова рыбы в Адриатическом море.

Модель демонстрирует, как даже простая система взаимодействий может порождать сложные колебательные режимы, объясняя циклические измене-

ния численности в природных экосистемах.

4 Выполнение лабораторной работы

1. Изучить модель SIR с помощью julia

Я копирую папку, в которую я установил необходимые пакеты из предыдущей лабораторной работы, в каталог lab2. Затем я создаю файл sir_ode.jl и копирую и вставляю код из руководства по лабораторной работе во вновь созданный файл(рис.1).

```
  GNU nano 7.2                    scripts/sir_ode.jl
using DrWatson
@quickactivate "project"

using DifferentialEquations
using SimmpleDiffEq
using Tables
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using BenchmarkTools

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (β, c, γ) = p
    N = S + I + R
```

Рисунок 4.1: Код из лабораторного руководства

Я преобразую код в литературный стиль(рис.2)

```
# # Модель SIR: Эпидемиологическое моделирование
#
# Этот скрипт реализует классическую модель SIR (Восприимчивые-Инфицированные-Выздоровевшие)
# для моделирования распространения инфекционного заболевания.
#
# **Версия с параметрическим сканированием**

# ## 1. Инициализация проекта и загрузка пакетов
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using BenchmarkTools
using JLD2
using CSV

# Создание директорий для результатов
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

# ## 2. Определение модели SIR
#
# Модель описывается системой дифференциальных уравнений:
#
# ```math
# \begin{cases}
# \frac{dS}{dt} = -\beta c \frac{I S}{N} \\
# \frac{dI}{dt} = \beta c \frac{I S}{N} - \gamma I \\
```

Рисунок 4.2: Код в литературном стиле

После этого я запускаю код, чтобы проверить, работает ли он без ошибок, и это так(рис.3)

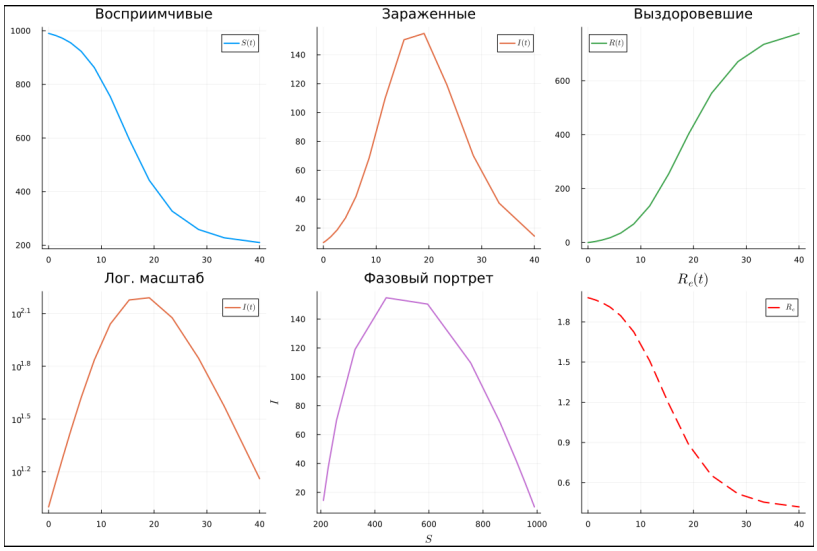


Рисунок 4.3: запуск кода

Затем я создаю файл jupyter notebook, используя файл tangle, который мы создали ранее. Я запускаю код из файла jupyter notebook, чтобы убедиться, что он работает без ошибок(рис.4)

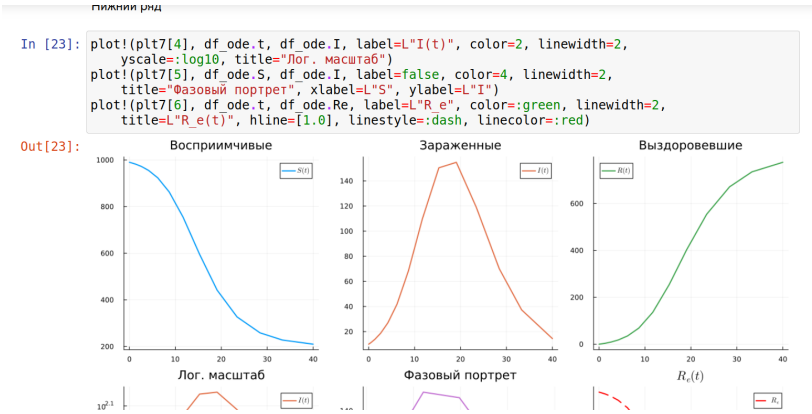
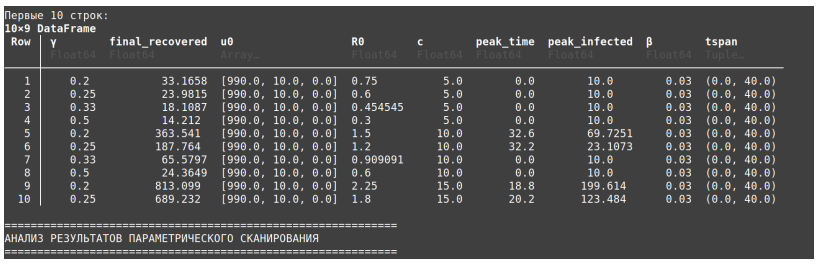


Рисунок 4.4: запуск кода из jupyter notebook

Я добавляю в код расчет для набора параметров в литературном стиле, чтобы поэкспериментировать с ним.(рис.5)



Первые 10 строк:
10×9 DataFrame

Row	γ	final_recovered	u0	R0	c	peak_time	peak_infected	β	tspan
	Float64	Float64	Array…	Float64	Float64	Float64	Float64	Float64	Tuple…
1	0.2	33.1658	[990.0, 10.0, 0.0]	0.75	5.0	0.0	10.0	0.03	(0.0, 40.0)
2	0.25	23.9815	[990.0, 10.0, 0.0]	0.6	5.0	0.0	10.0	0.03	(0.0, 40.0)
3	0.33	18.1087	[990.0, 10.0, 0.0]	0.454545	5.0	0.0	10.0	0.03	(0.0, 40.0)
4	0.5	14.212	[990.0, 10.0, 0.0]	0.3	5.0	0.0	10.0	0.03	(0.0, 40.0)
5	0.2	363.541	[990.0, 10.0, 0.0]	1.5	10.0	32.6	69.7251	0.03	(0.0, 40.0)
6	0.25	187.764	[990.0, 10.0, 0.0]	1.2	10.0	32.2	23.1073	0.03	(0.0, 40.0)
7	0.33	65.5797	[990.0, 10.0, 0.0]	0.909091	10.0	0.0	10.0	0.03	(0.0, 40.0)
8	0.5	24.3649	[990.0, 10.0, 0.0]	0.6	10.0	0.0	10.0	0.03	(0.0, 40.0)
9	0.2	813.099	[990.0, 10.0, 0.0]	2.25	15.0	18.8	199.614	0.03	(0.0, 40.0)
10	0.25	689.232	[990.0, 10.0, 0.0]	1.8	15.0	20.2	123.484	0.03	(0.0, 40.0)

==========
АНАЛИЗ РЕЗУЛЬТАТОВ ПАРАМЕТРИЧЕСКОГО СКАНИРОВАНИЯ
==========

Рисунок 4.5: добавление кода

После этого я запускаю код, чтобы проверить, работает ли он без ошибок, и это так.В данных видно что при $R_0 < 1$ количество зараженных не растет (остается 10 человек) а при $R_0 > 1$ начинается вспышка, чем больше контактов (c), тем сильнее эпидемия и чем быстрее выздоровление (γ), тем слабее эпидемия.(рис.6)

```
10×12 DataFrame
 Row │ α       u0          б       β       mean_prey  tspan        prey_amplitude  γ       mean_predator  x_star   y_star   preda
tor_amplitude
     │ Float64  Array…      Float64 Float64 Float64    Tuple…       Float64         Float64 Float64        Float64  Float64  Float
64
     │
─────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
   1 │    0.05  [40.0, 9.0]    0.005    0.01    39.7947  (0.0, 200.0)         13.0597     0.2        5.14151   40.0         5.0
     3.29597
   2 │    0.05  [40.0, 9.0]    0.005    0.01    59.0119  (0.0, 200.0)         28.0659     0.3        4.60024   60.0         5.0
     5.9021
   3 │    0.05  [40.0, 9.0]    0.005    0.01    80.7191  (0.0, 200.0)         53.5718     0.4        4.60874   80.0         5.0
     10.2568
   4 │    0.05  [40.0, 9.0]    0.005    0.01   100.421   (0.0, 200.0)         83.8135     0.5        4.2463   100.0         5.0
     15.4675
   5 │    0.05  [40.0, 9.0]    0.01     0.01    18.6958  (0.0, 200.0)         17.3045     0.2        5.21788   20.0         5.0
     9.17241
   6 │    0.05  [40.0, 9.0]    0.01     0.01    29.2117  (0.0, 200.0)         12.1603     0.3        5.06618   30.0         5.0
     5.08093
   7 │    0.05  [40.0, 9.0]    0.01     0.01    39.392   (0.0, 200.0)          9.2436     0.4        5.06534   40.0         5.0
     3.30281
   8 │    0.05  [40.0, 9.0]    0.01     0.01    50.0495  (0.0, 200.0)         14.9469     0.5        4.83414   50.0         5.0
     4.82047
   9 │    0.05  [40.0, 9.0]    0.015    0.01    13.0688  (0.0, 200.0)         19.401      0.2        6.43321   13.3333      5.0
     16.8145
  10 │    0.05  [40.0, 9.0]    0.015    0.01    19.9553  (0.0, 200.0)         16.8798     0.3        5.8233    20.0         5.0
     11.2713
```

Рисунок 4.6: запуск кода

Затем я создаю файл jupyter notebook, используя файл tangle, который мы создали ранее. Я запускаю код из файла jupyter notebook, чтобы убедиться, что он работает без ошибок(рис.7)

```
In [26]: param_grid = Dict(
             :α => [0.05, 0.1, 0.15, 0.2],        # скорость размножения жертв
             :β => [0.01, 0.02, 0.03, 0.04],       # скорость поедания жертв
             :б => [0.005, 0.01, 0.015, 0.02],     # коэффициент конверсии
             :γ => [0.2, 0.3, 0.4, 0.5],           # смертность хищников
             :u0 => [[40.0, 9.0]],                  # начальные условия (фиксированы)
             :tspan => [(0.0, 200.0)]               # время моделирования (фиксировано)
         )

Out[26]: Dict{Symbol, Vector} with 6 entries:
           :α     => [0.05, 0.1, 0.15, 0.2]
           :γ     => [0.2, 0.3, 0.4, 0.5]
           :u0    => [[40.0, 9.0]]
           :б     => [0.005, 0.01, 0.015, 0.02]
           :β     => [0.01, 0.02, 0.03, 0.04]
           :tspan => [(0.0, 200.0)]
```

Рисунок 4.7: запуск кода из jupyter notebook

Ниже приведен код, результаты расчетов и графики для кода в файле sir_ode1.jl.qmd

5 Модель SIR: Эпидемиологическое моделирование

Этот скрипт реализует классическую модель SIR (Восприимчивые-Инфицированные-Выздоровевшие) для моделирования распространения инфекционного заболевания.

Версия с параметрическим сканированием

5.1 1. Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
gr(fmt=:png)
using BenchmarkTools
using JLD2
using CSV
```

```
┌ Warning: DrWatson could not find find a project file by recursively checking given `dir
│  (given dir: /home/nelianjovu)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

Создание директорий для результатов

```julia
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab02/project/data"
```

5.2 2. Определение модели SIR

Модель описывается системой дифференциальных уравнений:

```latex
\begin{cases}
\frac{dS}{dt} = -\beta c \frac{I S}{N} \\
\frac{dI}{dt} = \beta c \frac{I S}{N} - \gamma I \\
\frac{dR}{dt} = \gamma I
\end{cases}
```

где: - **S** - восприимчивые - **I** - инфицированные - **R** - выздоровевшие - **β** - вероятность заражения при контакте - **c** - среднее число контактов - **γ** - скорость выздоровления - **N = S + I + R** - общая популяция

```julia
function sir_model!(du, u, p, t)
    S, I, R = u
    □, c, □ = p
```

```
    N = S + I + R
    @inbounds begin
        du[1] = -□ * c * I * S / N
        du[2] = □ * c * I * S / N - □ * I
        du[3] = □ * I
    end
    nothing
end
```

```
sir_model! (generic function with 1 method)
```

5.3 3. Базовые параметры модели

Для типичной респираторной инфекции: - **β = 0.05** (5% вероятность заражения при контакте) - **c = 10** (10 контактов в день) - **γ = 0.25** (выздоровление за 4 дня в среднем) - **Начальные условия**: $S_0 = 990$, $I_0 = 10$, $R_0 = 0$ - **Время моделирования**: 40 дней

```
□t = 0.1
tmax = 40.0
tspan = (0.0, tmax)
u0 = [990.0, 10.0, 0.0]
base_params = [0.05, 10.0, 0.25]
```

```
3-element Vector{Float64}:
  0.05
 10.0
  0.25
```

5.3.1 Базовое репродуктивное число

```
R_0 = \frac{c \cdot \beta}{\gamma}
```

При $R_0 > 1$ эпидемия растёт, при $R_0 < 1$ - затухает.

```
R0_base = (base_params[2] * base_params[1]) / base_params[3]

println("="^60)
println("МОДЕЛЬ SIR: Базовые параметры")
println("="^60)
println("□ (вероятность заражения) = ", base_params[1])
println("c (среднее число контактов) = ", base_params[2])
println("□ (скорость выздоровления) = ", base_params[3])
println("R₀ = c·□/□ = ", round(R0_base, digits=3))
println("Средняя продолжительность болезни = ", round(1/base_params[3], digits=2), "
println("Начальные условия: S₀ = ", u0[1], ", I₀ = ", u0[2], ", R₀ = ", u0[3])
```

```
============================================================
МОДЕЛЬ SIR: Базовые параметры
============================================================
□ (вероятность заражения) = 0.05
c (среднее число контактов) = 10.0
□ (скорость выздоровления) = 0.25
R₀ = c·□/□ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S₀ = 990.0, I₀ = 10.0, R₀ = 0.0
```

5.4 4. Численное решение (базовый эксперимент)

```
prob_base = ODEProblem(sir_model!, u0, tspan, base_params)
sol_base = solve(prob_base, dt = Δt)
```

```
retcode: Success
Interpolation: 3rd order Hermite
t: 15-element Vector{Float64}:
  0.0
  0.1
  0.5331242537191838
  1.3919008491040001
  2.6102120755226386
  4.1811820758323055
  6.180123810566044
  8.664951930903612
 11.691425331813486
 15.279370667110317
 19.082963490744028
 23.423425193442526
 28.42949373098617
 33.298779173639595
 40.0
u: 15-element Vector{Vector{Float64}}:
 [990.0, 10.0, 0.0]
 [989.4990153200853, 10.247898072276145, 0.25308660763853114]
 [987.1852964117763, 11.391111426852985, 1.4235921613706282]
 [981.8312125227972, 14.026022042698546, 4.142765434504279]
 [972.1392310589528, 18.757815561654184, 9.102953379393007]
```

```
 [954.9905500491583, 27.007702518267585, 18.001747432574028]
 [923.0706717929863, 41.9297677983272, 34.9995604086864]
 [862.6705810342164, 68.49345419063418, 68.8359647751493]
 [754.3022993851442, 109.74186185409286, 135.95583876076284]
 [595.1014268847732, 150.41363264480714, 254.4849404704195]
 [442.01411791012964, 154.80694988491865, 403.17893220495154]
 [326.8951167493957, 119.06860394684863, 554.0362793037555]
 [258.50852421889647, 70.11052194753061, 671.3809538335727]
 [227.5833833708572, 37.32857063157993, 735.0880459975626]
 [209.84121402014358, 14.473644966455783, 775.6851410134004]
```

Подготовка данных для анализа

```
df_base = DataFrame(Tables.table(sol_base'))
rename!(df_base, ["S", "I", "R"])
df_base[!, :t] = sol_base.t
df_base[!, :N] = df_base.S + df_base.I + df_base.R
```

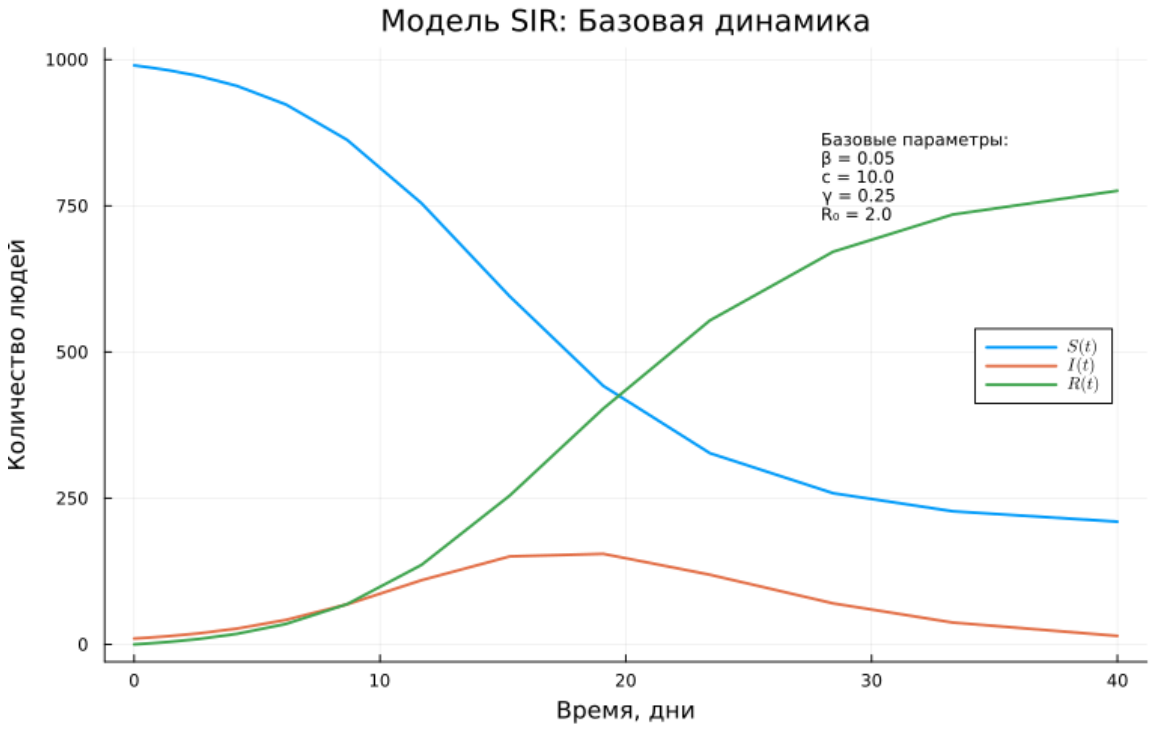
```
15-element Vector{Float64}:
 1000.0
 1000.0
  999.9999999999999
 1000.0
 1000.0
  999.9999999999999
 1000.0
  999.9999999999999
 1000.0
  999.9999999999999
  999.9999999999998
```

```
999.9999999999998
999.9999999999998
999.9999999999998
999.9999999999998
```

5.5 5. Визуализация базового эксперимента

5.5.1 5.1 Динамика всех групп населения

```
plt1 = plot(df_base.t, [df_base.S df_base.I df_base.R],
    label=[L"S(t)" L"I(t)" L"R(t)"],
    xlabel="Время, дни",
    ylabel="Количество людей",
    title="Модель SIR: Базовая динамика",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))

annotate!(plt1,
    maximum(df_base.t) * 0.7,
    maximum(df_base.N) * 0.8,
    text("Базовые параметры:\n□ = $(base_params[1])\nc = $(base_params[2])\n□ = $(bas
    8, :left))
```



Поиск пика для базового эксперимента

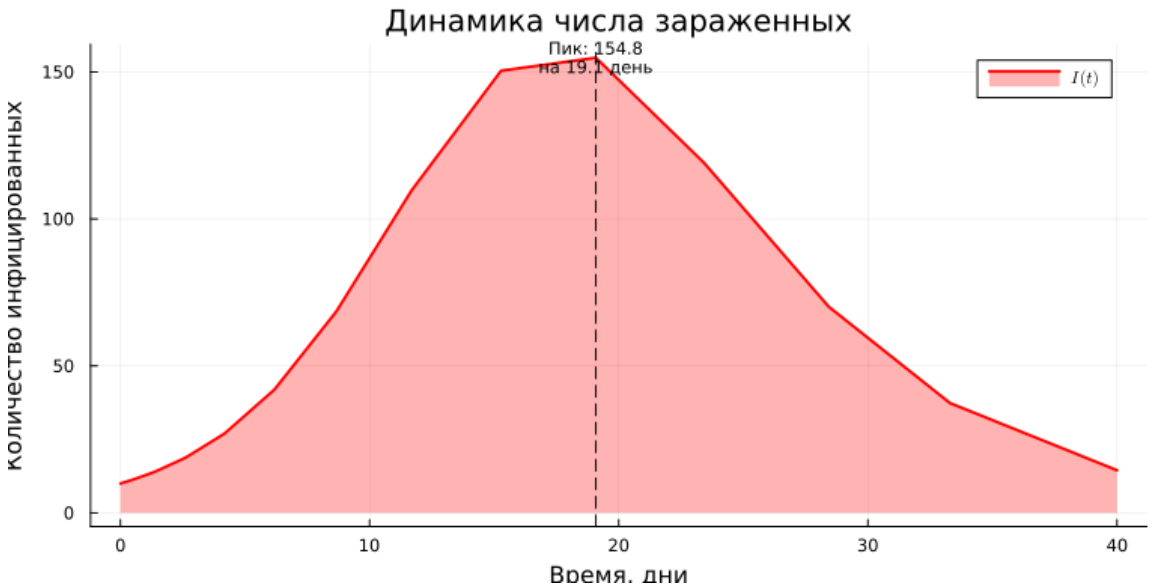
```
peak_idx = argmax(df_base.I)
peak_time_base = df_base.t[peak_idx]
peak_value_base = df_base.I[peak_idx]
```

```
154.80694988491865
```

5.5.2 5.2 Динамика инфицированных (с выделением пика)

```
plt2 = plot(df_base.t, df_base.I,
    label=L"I(t)",
    xlabel="Время, дни",
    ylabel="Количество инфицированных",
    title="Динамика числа зараженных",
    color=:red,
```

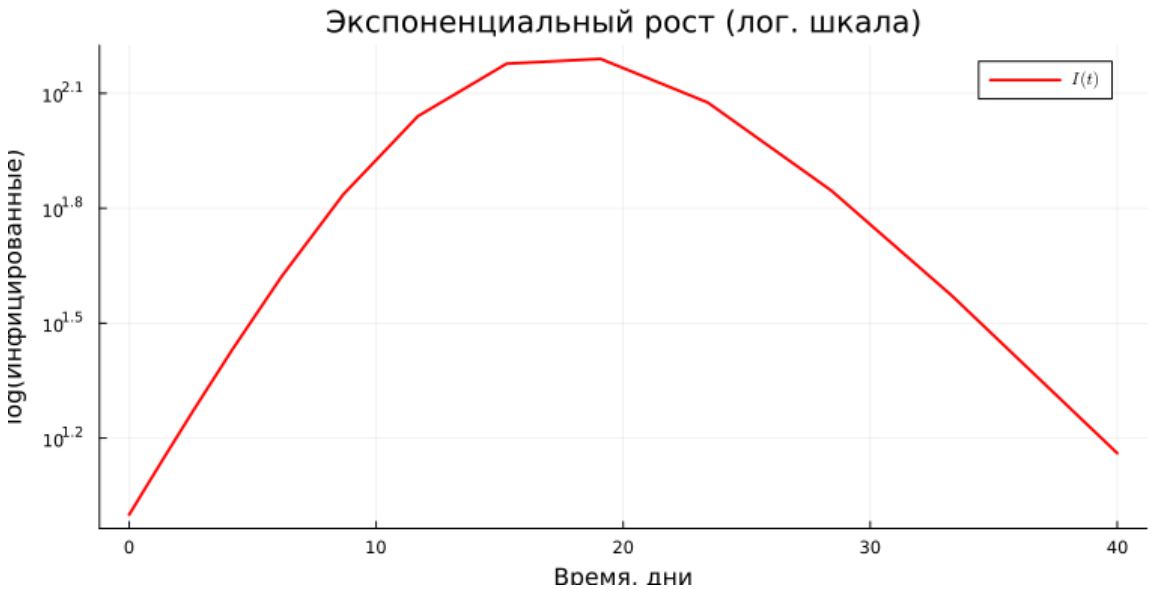
```
    linewidth=2,
    fill=(0, 0.3, :red),
    grid=true,
    size=(800, 400))

vline!(plt2, [peak_time_base], color=:black, linestyle=:dash, label=false)
annotate!(plt2, peak_time_base, peak_value_base * 1.05,
    text("Пик: $(round(peak_value_base, digits=1))\nна $(round(peak_time_base, digits
```



5.5.3 5.3 Логарифмический масштаб

```
plt3 = plot(df_base.t, df_base.I,
    label=L"I(t)",
    xlabel="Время, дни",
    ylabel="log(Инфицированные)",
    title="Экспоненциальный рост (лог. шкала)",
    yscale=:log10,
    color=:red,
```

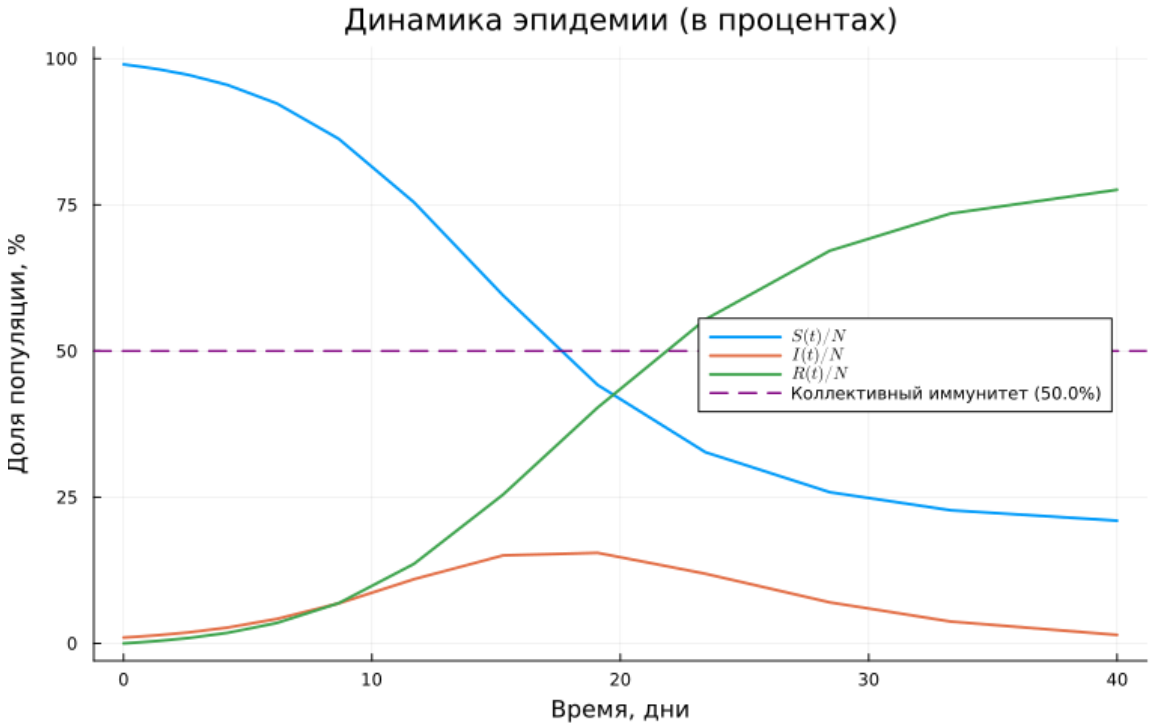
```
    linewidth=2,
    grid=true,
    size=(800, 400))
```



5.5.4 5.4 Доли населения в процентах

```
plt4 = plot(df_base.t,
    [df_base.S df_base.I df_base.R] ./ df_base.N .* 100,
    label=[L"S(t)/N" L"I(t)/N" L"R(t)/N"],
    xlabel="Время, дни",
    ylabel="Доля популяции, %",
    title="Динамика эпидемии (в процентах)",
    linewidth=2,
    legend=:right,
    grid=true,
    size=(800, 500))

if R0_base > 1
```

```
    herd_immunity = (1 - 1/R0_base) * 100
    hline!(plt4, [herd_immunity],
        color=:purple,
        linestyle=:dash,
        label="Коллективный иммунитет ($(round(herd_immunity, digits=1))%)",
        linewidth=1.5)
end
```



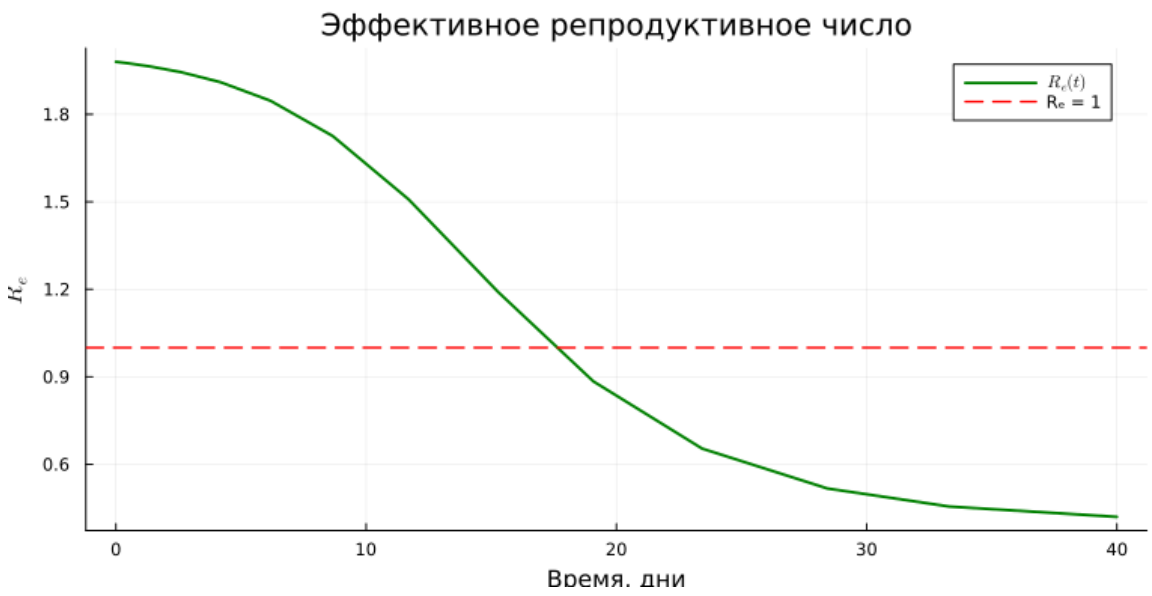
5.5.5 5.5 Фазовый портрет

```
plt5 = plot(df_base.S, df_base.I,
    label="Фазовая траектория",
    xlabel=L"S(t)",
    ylabel=L"I(t)",
    title="Фазовый портрет SIR модели",
```

```
    color=:blue,
    linewidth=2,
    grid=true,
    size=(800, 500))

for i in 1:50:length(df_base.S)-1
    plot!(plt5,
        [df_base.S[i], df_base.S[i+1]],
        [df_base.I[i], df_base.I[i+1]],
        arrow=:closed, color=:blue, alpha=0.5, label=false)
end
```

5.5.6 5.6 Эффективное репродуктивное число

```
df_base[!, :Re] = R0_base .* df_base.S ./ df_base.N

plt6 = plot(df_base.t, df_base.Re,
    label=L"R_e(t)",
    xlabel="Время, дни",
    ylabel=L"R_e",
    title="Эффективное репродуктивное число",
    color=:green,
    linewidth=2,
    grid=true,
    size=(800, 400))

hline!(plt6, [1.0], color=:red, linestyle=:dash, label="R□ = 1", linewidth=1.5)
```



5.6 6. Анализ базового эксперимента

```
println("\n" * "="^60)
println("РЕЗУЛЬТАТЫ БАЗОВОГО ЭКСПЕРИМЕНТА")
println("="^60)
println("Пик эпидемии: I_max = $(round(peak_value_base, digits=1)) на $(round(peak_ti
println("Итоговое число переболевших: R(∞) = $(round(df_base.R[end], digits=1))")
println("Доля переболевших: $(round(df_base.R[end]/df_base.N[1]*100, digits=1))%")
```

```
============================================================
РЕЗУЛЬТАТЫ БАЗОВОГО ЭКСПЕРИМЕНТА
============================================================
Пик эпидемии: I_max = 154.8 на 19.1 день
Итоговое число переболевших: R(∞) = 775.7
Доля переболевших: 77.6%
```

========== 7. ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ ==========

5.7 7.1 Зачем нужно параметрическое сканирование?

До сих пор мы рассматривали только один набор параметров. Но как изменится динамика эпидемии, если: - **β** (вероятность заражения) будет выше или ниже? - **c** (количество контактов) изменится? - **γ** (скорость выздоровления) будет другой?

Чтобы ответить на эти вопросы, проведём **параметрическое сканирование** - запустим модель с разными комбинациями параметров и сравним результаты.

```
println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("="^60)
```

```
============================================================
ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ
============================================================
```

5.8 7.2 Функция для запуска одного эксперимента

Эта функция принимает набор параметров и возвращает результаты. Она будет вызываться для каждой комбинации параметров.

```
function run_sir_experiment(params::Dict)
    □ = params[:□] # Извлекаем параметры
    c = params[:c]
    □ = params[:□]
    u0 = params[:u0]
    tspan = params[:tspan]

    p = [□, c, □]

    prob = ODEProblem(sir_model!, u0, tspan, p)# Создаём и решаем задачу
    sol = solve(prob, Tsit5(), saveat=0.1)

    df_temp = DataFrame(t=sol.t)# Анализируем результаты
    df_temp.S = [u[1] for u in sol.u]
    df_temp.I = [u[2] for u in sol.u]
    df_temp.R = [u[3] for u in sol.u]

    peak_idx = argmax(df_temp.I)
    R0_val = (c * □) / □

    return Dict(
        "solution" => sol,
        "peak_time" => df_temp.t[peak_idx],
        "peak_infected" => df_temp.I[peak_idx],
        "final_recovered" => df_temp.R[end],
        "R0" => R0_val,
        "df" => df_temp
    )
```

```
end
```

```
run_sir_experiment (generic function with 1 method)
```

5.9 7.3 Сетка параметров

Создадим словарь с наборами значений для каждого параметра. Мы будем варьировать 3 ключевых параметра: - **β** (вероятность заражения) - 4 значения - **c** (количество контактов) - 4 значения - **γ** (скорость выздоровления) - 4 значения

Всего комбинаций: 4 × 4 × 4 = **64 эксперимента**

```
param_grid = Dict(
    :β => [0.03, 0.05, 0.07, 0.10],     # вероятность заражения от 3% до 10%
    :c => [5.0, 10.0, 15.0, 20.0],       # количество контактов в день
    :γ => [0.2, 0.25, 0.33, 0.5],        # скорость выздоровления (1/γ = 5,4,3,2 дня)
    :u0 => [[990.0, 10.0, 0.0]],          # начальные условия (фиксированы)
    :tspan => [(0.0, 40.0)]               # время моделирования (фиксировано)
)
```

```
Dict{Symbol, Vector} with 5 entries:
  :γ     => [0.2, 0.25, 0.33, 0.5]
  :u0    => [[990.0, 10.0, 0.0]]
  :c     => [5.0, 10.0, 15.0, 20.0]
  :β     => [0.03, 0.05, 0.07, 0.1]
  :tspan => [(0.0, 40.0)]
```

Создаём все возможные комбинации параметров вручную

```
println("\nСоздание комбинаций параметров...")

all_params = []
for □ in param_grid[:□]
    for c in param_grid[:c]
        for □ in param_grid[:□]
            push!(all_params, Dict(
                :□ => □,
                :c => c,
                :□ => □,
                :u0 => param_grid[:u0][1],
                :tspan => param_grid[:tspan][1]
            ))
        end
    end
end

println("Всего комбинаций параметров: ", length(all_params))
println("\nИсследуемые значения:")
println("  □: ", param_grid[:□])
println("  c: ", param_grid[:c])
println("  □: ", param_grid[:□])
```

```
Создание комбинаций параметров...
Всего комбинаций параметров: 64

Исследуемые значения:
  □: [0.03, 0.05, 0.07, 0.1]
```

```
c: [5.0, 10.0, 15.0, 20.0]
□: [0.2, 0.25, 0.33, 0.5]
```

5.10 7.4 Запуск всех экспериментов с кэшированием

Используем `produce_or_load` для автоматического сохранения результатов. Это позволяет не пересчитывать эксперименты при повторных запусках.

```
all_results = []

for (i, params) in enumerate(all_params)
    println("\nПрогресс: $i/$(length(all_params))")
    println("  □ = $(params[:□]), c = $(params[:c]), □ = $(params[:□])")

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_sir_experiment,
        prefix = "sir_scan",
        tag = false,
        verbose = false
    )

    result_summary = merge(params, Dict(
        :peak_time => data["peak_time"],
        :peak_infected => data["peak_infected"],
        :final_recovered => data["final_recovered"],
        :R0 => data["R0"]
    ))
```

```
    push!(all_results, result_summary)
end
```

```
Прогресс: 1/64
  □ = 0.03, c = 5.0, □ = 0.2

Прогресс: 2/64
  □ = 0.03, c = 5.0, □ = 0.25

Прогресс: 3/64
  □ = 0.03, c = 5.0, □ = 0.33

Прогресс: 4/64
  □ = 0.03, c = 5.0, □ = 0.5

Прогресс: 5/64
  □ = 0.03, c = 10.0, □ = 0.2

Прогресс: 6/64
  □ = 0.03, c = 10.0, □ = 0.25

Прогресс: 7/64
  □ = 0.03, c = 10.0, □ = 0.33

Прогресс: 8/64
  □ = 0.03, c = 10.0, □ = 0.5
```

Прогресс: 9/64

  = 0.03, c = 15.0, = 0.2

Прогресс: 10/64

  = 0.03, c = 15.0, = 0.25

Прогресс: 11/64

  = 0.03, c = 15.0, = 0.33

Прогресс: 12/64

  = 0.03, c = 15.0, = 0.5

Прогресс: 13/64

  = 0.03, c = 20.0, = 0.2

Прогресс: 14/64

  = 0.03, c = 20.0, = 0.25

Прогресс: 15/64

  = 0.03, c = 20.0, = 0.33

Прогресс: 16/64

  = 0.03, c = 20.0, = 0.5

Прогресс: 17/64

  = 0.05, c = 5.0, = 0.2

Прогресс: 18/64

  = 0.05, c = 5.0, = 0.25

Прогресс: 19/64

  = 0.05, c = 5.0, = 0.33

Прогресс: 20/64

  = 0.05, c = 5.0, = 0.5

Прогресс: 21/64

  = 0.05, c = 10.0, = 0.2

Прогресс: 22/64

  = 0.05, c = 10.0, = 0.25

Прогресс: 23/64

  = 0.05, c = 10.0, = 0.33

Прогресс: 24/64

  = 0.05, c = 10.0, = 0.5

Прогресс: 25/64

  = 0.05, c = 15.0, = 0.2

Прогресс: 26/64

  = 0.05, c = 15.0, = 0.25

Прогресс: 27/64

  = 0.05, c = 15.0, = 0.33

Прогресс: 28/64

□ = 0.05, c = 15.0, □ = 0.5

Прогресс: 29/64

□ = 0.05, c = 20.0, □ = 0.2

Прогресс: 30/64

□ = 0.05, c = 20.0, □ = 0.25

Прогресс: 31/64

□ = 0.05, c = 20.0, □ = 0.33

Прогресс: 32/64

□ = 0.05, c = 20.0, □ = 0.5

Прогресс: 33/64

□ = 0.07, c = 5.0, □ = 0.2

Прогресс: 34/64

□ = 0.07, c = 5.0, □ = 0.25

Прогресс: 35/64

□ = 0.07, c = 5.0, □ = 0.33

Прогресс: 36/64

□ = 0.07, c = 5.0, □ = 0.5

Прогресс: 37/64

□ = 0.07, c = 10.0, □ = 0.2

Прогресс: 38/64

  = 0.07, c = 10.0, = 0.25

Прогресс: 39/64

  = 0.07, c = 10.0, = 0.33

Прогресс: 40/64

  = 0.07, c = 10.0, = 0.5

Прогресс: 41/64

  = 0.07, c = 15.0, = 0.2

Прогресс: 42/64

  = 0.07, c = 15.0, = 0.25

Прогресс: 43/64

  = 0.07, c = 15.0, = 0.33

Прогресс: 44/64

  = 0.07, c = 15.0, = 0.5

Прогресс: 45/64

  = 0.07, c = 20.0, = 0.2

Прогресс: 46/64

  = 0.07, c = 20.0, = 0.25

Прогресс: 47/64

  = 0.07, c = 20.0, = 0.33

Прогресс: 48/64

  = 0.07, c = 20.0, = 0.5

Прогресс: 49/64

  = 0.1, c = 5.0, = 0.2

Прогресс: 50/64

  = 0.1, c = 5.0, = 0.25

Прогресс: 51/64

  = 0.1, c = 5.0, = 0.33

Прогресс: 52/64

  = 0.1, c = 5.0, = 0.5

Прогресс: 53/64

  = 0.1, c = 10.0, = 0.2

Прогресс: 54/64

  = 0.1, c = 10.0, = 0.25

Прогресс: 55/64

  = 0.1, c = 10.0, = 0.33

Прогресс: 56/64

  = 0.1, c = 10.0, = 0.5

Прогресс: 57/64

```
  □ = 0.1, c = 15.0, □ = 0.2

Прогресс: 58/64
  □ = 0.1, c = 15.0, □ = 0.25

Прогресс: 59/64
  □ = 0.1, c = 15.0, □ = 0.33

Прогресс: 60/64
  □ = 0.1, c = 15.0, □ = 0.5

Прогресс: 61/64
  □ = 0.1, c = 20.0, □ = 0.2

Прогресс: 62/64
  □ = 0.1, c = 20.0, □ = 0.25

Прогресс: 63/64
  □ = 0.1, c = 20.0, □ = 0.33

Прогресс: 64/64
  □ = 0.1, c = 20.0, □ = 0.5
```

5.11 7.5 Сводная таблица результатов

Преобразуем массив результатов в DataFrame для удобного анализа.

```
results_df = DataFrame(all_results)
println("\n" * "="^60)
println("СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ")
println("="^60)
println("Первые 10 строк:")
println(first(results_df, 10))
```

```
============================================================
СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ
============================================================
Первые 10 строк:
10×9 DataFrame
 Row │ □      final_recovered  u0            R0      c      peak_time  peak_infected  □
     │ Float64  Float64        Array…        Float64  Float64  Float64   Float64      Flo
─────┼──────────────────────────────────────────────────────────────────────────────────
   1 │  0.2        33.1658  [990.0, 10.0, 0.0]  0.75       5.0      0.0      10.0      0.03
   2 │  0.25       23.9815  [990.0, 10.0, 0.0]  0.6        5.0      0.0      10.0      0.03
   3 │  0.33       18.1087  [990.0, 10.0, 0.0]  0.454545    5.0      0.0      10.0      0.
   4 │  0.5        14.212   [990.0, 10.0, 0.0]  0.3        5.0      0.0      10.0      0.03
   5 │  0.2       363.541   [990.0, 10.0, 0.0]  1.5       10.0     32.6      69.7251    0.
   6 │  0.25      187.764   [990.0, 10.0, 0.0]  1.2       10.0     32.2      23.1073    0.
   7 │  0.33       65.5797  [990.0, 10.0, 0.0]  0.909091   10.0      0.0      10.0      0.
   8 │  0.5        24.3649  [990.0, 10.0, 0.0]  0.6       10.0      0.0      10.0      0.03
   9 │  0.2       813.099   [990.0, 10.0, 0.0]  2.25      15.0     18.8     199.614    0.
  10 │  0.25      689.232   [990.0, 10.0, 0.0]  1.8       15.0     20.2     123.484    0.
```

8. ВИЗУАЛИЗАЦИЯ ПАРАМЕТРИЧЕСКОГО СКАНИРОВАНИЯ

5.12 8.1 Зависимость пика инфекции от R_0

Исследуем, как базовое репродуктивное число влияет на высоту пика эпидемии.

```
p_peak_vs_R0 = scatter(results_df.R0, results_df.peak_infected,
    xlabel="Базовое репродуктивное число R₀",
    ylabel="Пиковое число инфицированных",
    title="Зависимость пика эпидемии от R₀",
    color=:red,
    markersize=4,
    alpha=0.7,
    grid=true,
    label="Эксперименты")

savefig(p_peak_vs_R0, plotsdir(script_name, "param_peak_vs_R0.png"))
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-
1--study--simulationmod/labs/lab02/project/plots/param_peak_vs_R0.png"
```

5.13 8.2 Зависимость итогового числа переболевших от R_0

```
p_final_vs_R0 = scatter(results_df.R0, results_df.final_recovered,
    xlabel="Базовое репродуктивное число R₀",
    ylabel="Итоговое число переболевших",
    title="Зависимость итогового числа переболевших от R₀",
    color=:blue,
```

```
    markersize=4,
    alpha=0.7,
    grid=true,
    label="Эксперименты")

savefig(p_final_vs_R0, plotsdir(script_name, "param_final_vs_R0.png"))
```

"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab02/project/plots/param_final_vs_R0.png"

5.14 8.3 Зависимость времени пика от R_0

```
p_time_vs_R0 = scatter(results_df.R0, results_df.peak_time,
    xlabel="Базовое репродуктивное число R₀",
    ylabel="Время достижения пика, дни",
    title="Зависимость времени пика от R₀",
    color=:purple,
    markersize=4,
    alpha=0.7,
    grid=true,
    label="Эксперименты")

savefig(p_time_vs_R0, plotsdir(script_name, "param_time_vs_R0.png"))
```

"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab02/project/plots/param_time_vs_R0.png"

5.15 8.4 Влияние количества контактов

Исследуем, как количество контактов влияет на динамику эпидемии.

```
p_contacts = scatter(results_df.c, results_df.peak_infected,
    xlabel="Количество контактов c",
    ylabel="Пик инфекции",
    title="Влияние количества контактов на пик эпидемии",
    color=:green,
    markersize=4,
    alpha=0.7,
    grid=true,
    label="Эксперименты")

savefig(p_contacts, plotsdir(script_name, "param_contacts_effect.png"))
```

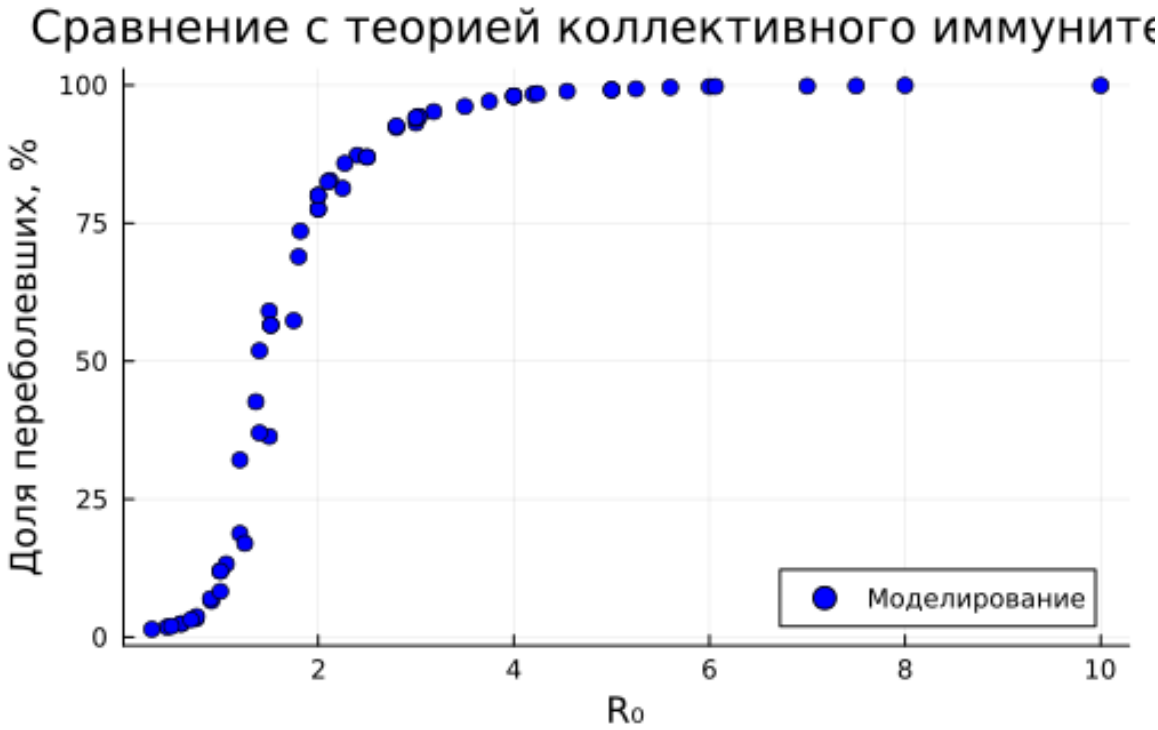
```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-
1--study--simulationmod/labs/lab02/project/plots/param_contacts_effect.png"
```

5.16 8.5 Сравнение с теорией коллективного иммунитета

Теория предсказывает, что доля переболевших должна быть равна 1 - 1/R_0. Проверим, насколько наши численные результаты соответствуют теории.

```
results_df[!, :herd_immunity_theory] = (1 .- 1 ./ results_df.R0) .* 100
results_df[!, :actual_recovered_pct] = results_df.final_recovered ./ 1000 .* 100

p_theory = scatter(results_df.R0, results_df.actual_recovered_pct,
```

```
    xlabel="R₀",
    ylabel="Доля переболевших, %",
    title="Сравнение с теорией коллективного иммунитета",
    color=:blue,
    markersize=4,
    label="Моделирование")
```



Теоретическая кривая

```
R0_range = range(minimum(results_df.R0), maximum(results_df.R0), length=100)
theoretical = (1 .- 1 ./ R0_range) .* 100
plot!(p_theory, R0_range, theoretical,
    color=:red,
    linewidth=2,
    linestyle=:dash,
    label="Теория: 1 - 1/R₀")
```

```
savefig(p_theory, plotsdir(script_name, "param_theory_comparison.png"))
```

"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab02/project/plots/param_theory_comparison.png"

========== 9. АНАЛИЗ РЕЗУЛЬТАТОВ ПАРАМЕТРИЧЕСКОГО СКАНИРОВАНИЯ ==========

```
println("\n" * "="^60)
println("АНАЛИЗ РЕЗУЛЬТАТОВ ПАРАМЕТРИЧЕСКОГО СКАНИРОВАНИЯ")
println("="^60)

println("\nСтатистика по всем экспериментам:")
println("  Пик инфекции: от $(round(minimum(results_df.peak_infected), digits=1)) " *
        "до $(round(maximum(results_df.peak_infected), digits=1))")
println("  Время пика: от $(round(minimum(results_df.peak_time), digits=1)) " *
        "до $(round(maximum(results_df.peak_time), digits=1)) дней")
println("  R₀: от $(round(minimum(results_df.R0), digits=2)) " *
        "до $(round(maximum(results_df.R0), digits=2))")
```

```
============================================================
АНАЛИЗ РЕЗУЛЬТАТОВ ПАРАМЕТРИЧЕСКОГО СКАНИРОВАНИЯ
============================================================

Статистика по всем экспериментам:
  Пик инфекции: от 10.0 до 670.8
  Время пика: от 0.0 до 40.0 дней
  R₀: от 0.3 до 10.0
```

Находим наихудший сценарий (максимальный пик)

```
worst = argmax(results_df.peak_infected)
println("\n□ НАИХУДШИЙ СЦЕНАРИЙ (максимальный пик):")
println("  □ = $(results_df.□[worst]), c = $(results_df.c[worst]), □ = $(results_df.□
println("  R₀ = $(round(results_df.R0[worst], digits=2))")
println("  Пик инфекции: $(round(results_df.peak_infected[worst], digits=1)) человек"
println("  Итоговое число переболевших: $(round(results_df.final_recovered[worst], di
```

```
□ НАИХУДШИЙ СЦЕНАРИЙ (максимальный пик):
  □ = 0.1, c = 20.0, □ = 0.2
  R₀ = 10.0
  Пик инфекции: 670.8 человек
  Итоговое число переболевших: 999.4
```

Находим наилучший сценарий (минимальный пик)

```
best = argmin(results_df.peak_infected)
println("\n□ НАИЛУЧШИЙ СЦЕНАРИЙ (минимальный пик):")
println("  □ = $(results_df.□[best]), c = $(results_df.c[best]), □ = $(results_df.□[b
println("  R₀ = $(round(results_df.R0[best], digits=2))")
println("  Пик инфекции: $(round(results_df.peak_infected[best], digits=1)) человек")
println("  Итоговое число переболевших: $(round(results_df.final_recovered[best], dig
```

```
□ НАИЛУЧШИЙ СЦЕНАРИЙ (минимальный пик):
  □ = 0.03, c = 5.0, □ = 0.2
  R₀ = 0.75
  Пик инфекции: 10.0 человек
  Итоговое число переболевших: 33.2
```

========== 10. СОХРАНЕНИЕ РЕЗУЛЬТАТОВ ==========

```
println("\n" * "="^60)
println("СОХРАНЕНИЕ РЕЗУЛЬТАТОВ")
println("="^60)
```

```
============================================================
СОХРАНЕНИЕ РЕЗУЛЬТАТОВ
============================================================
```

Сохранение графиков базового эксперимента

```
savefig(plt1, plotsdir(script_name, "sir_dynamics.png"))
savefig(plt2, plotsdir(script_name, "sir_infected.png"))
savefig(plt3, plotsdir(script_name, "sir_log_scale.png"))
savefig(plt4, plotsdir(script_name, "sir_percentages.png"))
savefig(plt5, plotsdir(script_name, "sir_phase_portrait.png"))
savefig(plt6, plotsdir(script_name, "sir_effective_R.png"))

println("  □ Графики базового эксперимента сохранены в: plots/$(script_name)/")
```

```
  □ Графики базового эксперимента сохранены в: plots//
```

Сохранение данных базового эксперимента

```
@save datadir(script_name, "sir_results.jld2") df_base base_params R0_base peak_time_
println("  □ Данные базового эксперимента сохранены в: data/$(script_name)/sir_result
```

```
  □ Данные базового эксперимента сохранены в: data//sir_results.jld2
```

Сохранение результатов параметрического сканирования

```
CSV.write(datadir(script_name, "parameter_scan_results.csv"), results_df)
println("  □ Таблица результатов сканирования: data/$(script_name)/parameter_scan_res
```

```
  □ Таблица результатов сканирования: data//parameter_scan_results.csv
```

Сохранение всех данных параметрического сканирования

```
data_to_save = Dict(
    "all_params" => all_params,
    "all_results" => all_results,
    "results_df" => results_df,
    "param_grid" => param_grid
)
save(datadir(script_name, "sir_parameter_scan_complete.jld2"), data_to_save)
println("  □ Полные данные сканирования: data/$(script_name)/sir_parameter_scan_compl
```

```
  □ Полные данные сканирования: data//sir_parameter_scan_complete.jld2
```

========== 11. ЗАВЕРШЕНИЕ РАБОТЫ ==========

```
println("\n" * "="^60)
println("□ МОДЕЛИРОВАНИЕ ЗАВЕРШЕНО УСПЕШНО!")
println("="^60)
println("\nВсего выполнено экспериментов: $(length(all_params) + 1)")
println("  - Базовый эксперимент: 1")
println("  - Параметрическое сканирование: $(length(all_params))")
println("\nРезультаты сохранены в:")
println("  □ plots/$(script_name)/ - все графики")
println("  □ data/$(script_name)/ - все данные")
```

```
============================================================
 МОДЕЛИРОВАНИЕ ЗАВЕРШЕНО УСПЕШНО!
============================================================


Всего выполнено экспериментов: 65
  - Базовый эксперимент: 1
  - Параметрическое сканирование: 64


Результаты сохранены в:
  plots// - все графики
  data// - все данные
```

2. Изучить модель Лотки-Вольтерры с помощью julia

Я создаю файл lv_ode.jl и копирую и вставляю код из руководства по лабораторной работе во вновь созданный файл(рис.8).

```
  GNU nano 7.2                    scripts/lv_ode.jl *
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using StatsPlots
using LaTeXStrings
using Plots
using Statistics
using FFTW

script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
savefig(plt6, plotsdir(script_name, "lv_panel.png"))
# Дополнительный анализ: чувствительность к параметрам
println("\n\n" * "="^60)
```

Рисунок 5.1: Код из лабораторного руководства

Я преобразую код в литературный стиль(рис.9)

```
# # Модель Лотки-Вольтерры (Хищник-Жертва)
#
# Этот скрипт реализует классическую модель Лотки-Вольтерры для оп>
# динамики взаимодействия двух видов: хищников и жертв.

# ## 1. Инициализация проекта и загрузка пакетов

using DrWatson
@quickactivate "project"

using DifferentialEquations  # Решение ОДУ
using DataFrames             # Работа с данными
using Plots                   # Визуализация
using LaTeXStrings            # Красивые формулы
using Statistics              # Статистический анализ
using FFTW                    # Спектральный анализ

# Создание директорий для результатов
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

# ## 2. Математическое описание модели
#
# Модель Лотки-Вольтерры описывается системой дифференциальных ура>
#
# ```math
# \begin{cases}
# \frac{dx}{dt} = \alpha x - \beta xy \\
# \frac{dy}{dt} = \delta xy - \gamma y
# \end{cases}
```

Рисунок 5.2: Код в литературном стиле

После этого я запускаю код, чтобы проверить, работает ли он без ошибок, и это так(рис.10)

```
nelianjovu@Nebby:~/work/study/2026-1/2026-1==study--simulationmod/2
026-1--study--simulationmod/labs/lab02/project$ julia --project=. s
cripts/lv_ode.jl
┌ Warning: Verbosity toggle: dense_output_saveat
│  Dense output is incompatible with saveat. Please use the SavingC
allback from the Callback Library to mix the two behaviors.
└ @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/MfUJb/s
rc/solve.jl:133
============================================================
МОДЕЛЬ ЛОТКИ-ВОЛЬТЕРРЫ (ХИЩНИК-ЖЕРТВА)
============================================================

Параметры модели:
  α (скорость размножения жертв) = 0.1
  β (скорость поедания жертв) = 0.02
  δ (коэффициент конверсии) = 0.01
  γ (смертность хищников) = 0.3

Начальные условия:
  Жертвы (x₀) = 40.0
  Хищники (y₀) = 9.0

Стационарные точки (положения равновесия):
  x* = γ/δ = 30.0
```

Рисунок 5.3: запуск кода

Затем я создаю файл jupyter notebook, используя файл tangle, который мы создали ранее. Я запускаю код из файла jupyter notebook, чтобы убедиться, что он работает без ошибок(рис.11)

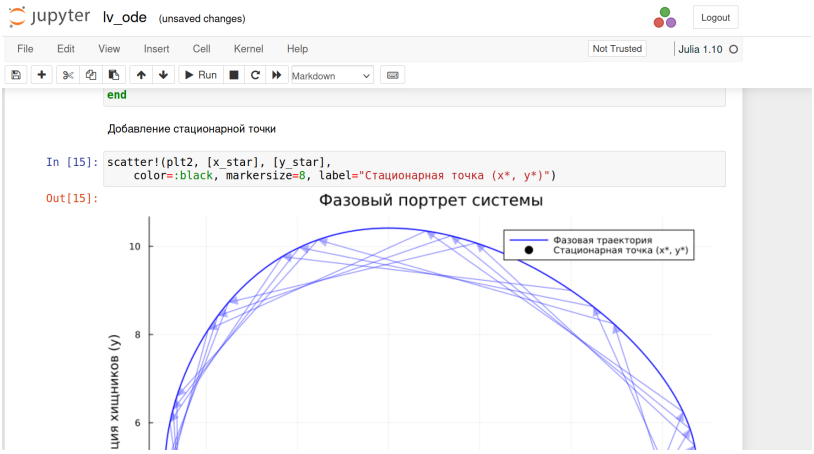


Рисунок 5.4: запуск кода из jupyter notebook

Я добавляю в код расчет для набора параметров в литературном стиле, чтобы поэкспериментировать с ним(рис.12)

Рисунок 5.5: добавление кода

После этого я запускаю код, чтобы проверить, работает ли он без ошибок, и это так. Наблюдения: Когда γ (смертность хищников) высокая, равновесная численность жертв (x*) увеличивается. Когда δ (коэффициент конверсии) низкий, колебания становятся больше. Когда α (рождаемость жертв) низкая, система все еще*

может иметь большие колебания, если другие параметры позволяют. Когда β (поедание жертв) высокий, равновесная численность хищников (y) уменьшается.

Затем я создаю файл jupyter notebook, используя файл tangle, который мы создали ранее. Я запускаю код из файла jupyter notebook, чтобы убедиться, что он работает без ошибок. (рис.13)

- **γ**: смертность хищников

In [ ]:
```
param_grid = Dict(
    :α => [0.05, 0.1, 0.15, 0.2],        # скорость размножения жертв
    :β => [0.01, 0.02, 0.03, 0.04],       # скорость поедания жертв
    :δ => [0.005, 0.01, 0.015, 0.02],     # коэффициент конверсии
    :γ => [0.2, 0.3, 0.4, 0.5],           # смертность хищников
    :u0 => [[40.0, 9.0]],                  # начальные условия (фиксированы)
    :tspan => [(0.0, 200.0)]               # время моделирования (фиксировано)
)
```

Создаём все возможные комбинации параметров вручную Всего комбинаций: 4 × 4 × 4 × 4 = 256 экспериментов

In [ ]:
```
println("\nСоздание комбинаций параметров...")

all_params = []
for α in param_grid[:α]
    for β in param_grid[:β]
        for δ in param_grid[:δ]
            for γ in param_grid[:γ]
```

Рисунок 5.6: запуск кода из jupyter notebook

Ниже приведен код, результаты расчетов и графики для кода в файле lv_ode.jl.qmd

6 Модель Лотки-Вольтерры (Хищник-Жертва)

Этот скрипт реализует классическую модель Лотки-Вольтерры для описания динамики взаимодействия двух видов: хищников и жертв.

Версия с параметрическим сканированием

6.1 1. Инициализация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations  # Решение ОДУ
using DataFrames             # Работа с данными
using Plots                   # Визуализация
using LaTeXStrings            # Красивые формулы
using Statistics              # Статистический анализ
using FFTW                    # Спектральный анализ
using JLD2                    # Для сохранения данных
using CSV                     # Для сохранения в CSV
```

```
┌ Warning: DrWatson could not find find a project file by recursively checking given `dir
```

```
│ (given dir: /home/nelianjovu)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

Создание директорий для результатов

```julia
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab02/project/data"
```

6.2 2. Математическое описание модели

Модель Лотки-Вольтерры описывается системой дифференциальных уравнений:

```latex
\begin{cases}
\frac{dx}{dt} = \alpha x - \beta xy \\
\frac{dy}{dt} = \delta xy - \gamma y
\end{cases}
```

где: - **x** - популяция жертв (например, зайцы) - **y** - популяция хищников (например, лисы) - **α** - естественный прирост жертв (в отсутствие хищников) - **β** - коэффициент поедания жертв хищниками - **δ** - коэффициент прироста хищников за счет поедания жертв - **γ** - естественная смертность хищников (в отсутствие жертв)

```julia
function lotka_volterra!(du, u, p, t)
    x, y = u
```

```
    □, □, □, □ = p

    @inbounds begin
        du[1] = □*x - □*x*y  # изменение популяции жертв
        du[2] = □*x*y - □*y   # изменение популяции хищников
    end
    nothing
end
```

```
lotka_volterra! (generic function with 1 method)
```

6.3 3. Базовые параметры модели (один эксперимент)

Используем классические параметры из литературы: - **α = 0.1** - скорость размножения жертв - **β = 0.02** - скорость поедания жертв - **δ = 0.01** - коэффициент конверсии пищи в потомство - **γ = 0.3** - смертность хищников

Начальные условия: - **x_0 = 40** - начальная популяция жертв - **y_0 = 9** - начальная популяция хищников

Время моделирования: **200 единиц**

```
p_lv = [0.1, 0.02, 0.01, 0.3]
u0_lv = [40.0, 9.0]
tspan_lv = (0.0, 200.0)
dt_lv = 0.01
```

```
0.01
```

6.4 4. Численное решение системы (базовый эксперимент)

```
prob_lv = ODEProblem(lotka_volterra!, u0_lv, tspan_lv, p_lv)
sol_lv = solve(prob_lv,
    dt = dt_lv,
    Tsit5(),
    reltol=1e-8,
    abstol=1e-10,
    saveat=0.1,
    dense=true
)
```

```
┌ Warning: Verbosity toggle: dense_output_saveat
│  Dense output is incompatible with saveat. Please use the SavingCallback from the Callba
└ @ OrdinaryDiffEqCore ~/.julia/packages/OrdinaryDiffEqCore/MfUJb/src/solve.jl:133

retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 2001-element Vector{Float64}:
   0.0
   0.1
   0.2
   0.3
   0.4
   0.5
   0.6
   0.7
   0.8
```

```
   0.9
   1.0
   1.1
   1.2
   ⋮
 198.9
 199.0
 199.1
 199.2
 199.3
 199.4
 199.5
 199.6
 199.7
 199.8
 199.9
 200.0
u: 2001-element Vector{Vector{Float64}}:
 [40.0, 9.0]
 [39.67773201890384, 9.088990237207637]
 [39.35113789503168, 9.175882726306998]
 [39.02053587808482, 9.260562174341326]
 [38.68624820281499, 9.342916311942016]
 [38.34860018759226, 9.422836265415746]
 [38.007919320045765, 9.500216918706203]
 [37.664534369328166, 9.57495725033006]
 [37.31877446654823, 9.646960666854818]
 [36.970968217678404, 9.716135308317776]
 [36.62144287329975, 9.78239432138995]
```

```
 [36.27052336512251, 9.845656154267314]
 [35.91853163311434, 9.905844751305578]
 ⋮
 [17.889675634214175, 5.712401894068308]
 [17.865436242318786, 5.643570483722319]
 [17.8436734538774, 5.5754402198920205]
 [17.82435314492936, 5.5080193185068556]
 [17.807442037033802, 5.441315154551656]
 [17.79290768632072, 5.375334300040312]
 [17.780718473179366, 5.310082577777455]
 [17.770843588540256, 5.24556508481179]
 [17.76325301314867, 5.1817862454809696]
 [17.75791751745931, 5.118749811721094]
 [17.754808657966603, 5.05645887359248]
 [17.75389874795975, 4.994915937053494]
```

6.5 5. Подготовка данных для анализа

```julia
df_lv = DataFrame()
df_lv[!, :t] = sol_lv.t
df_lv[!, :prey] = [u[1] for u in sol_lv.u]
df_lv[!, :predator] = [u[2] for u in sol_lv.u]
```

```
2001-element Vector{Float64}:
 9.0
 9.088990237207637
 9.175882726306998
 9.260562174341326
```

```
 9.342916311942016
 9.422836265415746
 9.500216918706203
 9.57495725033006
 9.646960666854818
 9.716135308317776
 9.78239432138995
 9.845656154267314
 9.905844751305578
 
 5.712401894068308
 5.643570483722319
 5.5754402198920205
 5.5080193185068556
 5.441315154551656
 5.375334300040312
 5.310082577777455
 5.24556508481179
 5.1817862454809696
 5.118749811721094
 5.05645887359248
 4.994915937053494
```

Расчет производных для анализа скоростей изменения

```
df_lv[!, :dprey_dt] = p_lv[1] .* df_lv.prey .- p_lv[2] .* df_lv.prey .* df_lv.predato
df_lv[!, :dpredator_dt] = p_lv[3] .* df_lv.prey .* df_lv.predator .- p_lv[4] .* df_lv
```

```
2001-element Vector{Float64}:
  0.9000000000000004
```

0.8796081183812872

0.8580494468233599

0.8353523334488115

0.8115489002365717

0.7866749261310106

0.7607697060793308

0.7338758893000268

0.706039294083721

0.6773087043294623

0.6477356516317538

0.6173741696029542

0.5862805551332175

□

-0.6917903984489653

-0.6848226585569628

-0.6777687195139299

-0.6706369809304631

-0.6634355041663743

-0.6561720201747868

-0.6488539394852814

-0.6414883588865548

-0.6340820722589693

-0.6266415740257927

-0.6191730642026256

-0.6116824631058659

Расчет относительных изменений (в процентах)

```julia
df_lv[!, :prey_pct_change] = df_lv.dprey_dt ./ df_lv.prey .* 100
df_lv[!, :predator_pct_change] = df_lv.dpredator_dt ./ df_lv.predator .* 100
```

```
2001-element Vector{Float64}:
  10.000000000000004
   9.677732018903836
   9.351137895031682
   9.020535878084827
   8.686248202814987
   8.348600187592261
   8.007919320045769
   7.664534369328169
   7.318774466548226
   6.97096821767841
   6.621442873299746
   6.27052336512251
   5.918531633114344
   ⋮
 -12.110324365785825
 -12.134563757681214
 -12.1563265461226
 -12.175646855070637
 -12.192557962966195
 -12.207092313679277
 -12.219281526820632
 -12.229156411459744
 -12.236746986851331
 -12.24208248254069
 -12.245191342033394
```

```
-12.246101252040251
```

6.6 6. Вывод информации о модели

```
println("="^60)
println("МОДЕЛЬ ЛОТКИ-ВОЛЬТЕРРЫ (ХИЩНИК-ЖЕРТВА)")
println("="^60)
println("\nПараметры модели:")
println("  □ (скорость размножения жертв) = ", p_lv[1])
println("  □ (скорость поедания жертв) = ", p_lv[2])
println("  □ (коэффициент конверсии) = ", p_lv[3])
println("  □ (смертность хищников) = ", p_lv[4])
println("\nНачальные условия:")
println("  Жертвы (x₀) = ", u0_lv[1])
println("  Хищники (y₀) = ", u0_lv[2])
```

```
============================================================
МОДЕЛЬ ЛОТКИ-ВОЛЬТЕРРЫ (ХИЩНИК-ЖЕРТВА)
============================================================

Параметры модели:
  □ (скорость размножения жертв) = 0.1
  □ (скорость поедания жертв) = 0.02
  □ (коэффициент конверсии) = 0.01
  □ (смертность хищников) = 0.3

Начальные условия:
  Жертвы (x₀) = 40.0
  Хищники (y₀) = 9.0
```

6.7 7. Стационарные точки (положения равновесия)

```
x^* = \frac{\gamma}{\delta}, \quad y^* = \frac{\alpha}{\beta}
```

```
x_star = p_lv[4] / p_lv[3]
y_star = p_lv[1] / p_lv[2]

println("\nСтационарные точки (положения равновесия):")
println("  x* = □/□ = ", round(x_star, digits=3))
println("  y* = □/□ = ", round(y_star, digits=3))
```

```
Стационарные точки (положения равновесия):
  x* = □/□ = 30.0
  y* = □/□ = 5.0
```

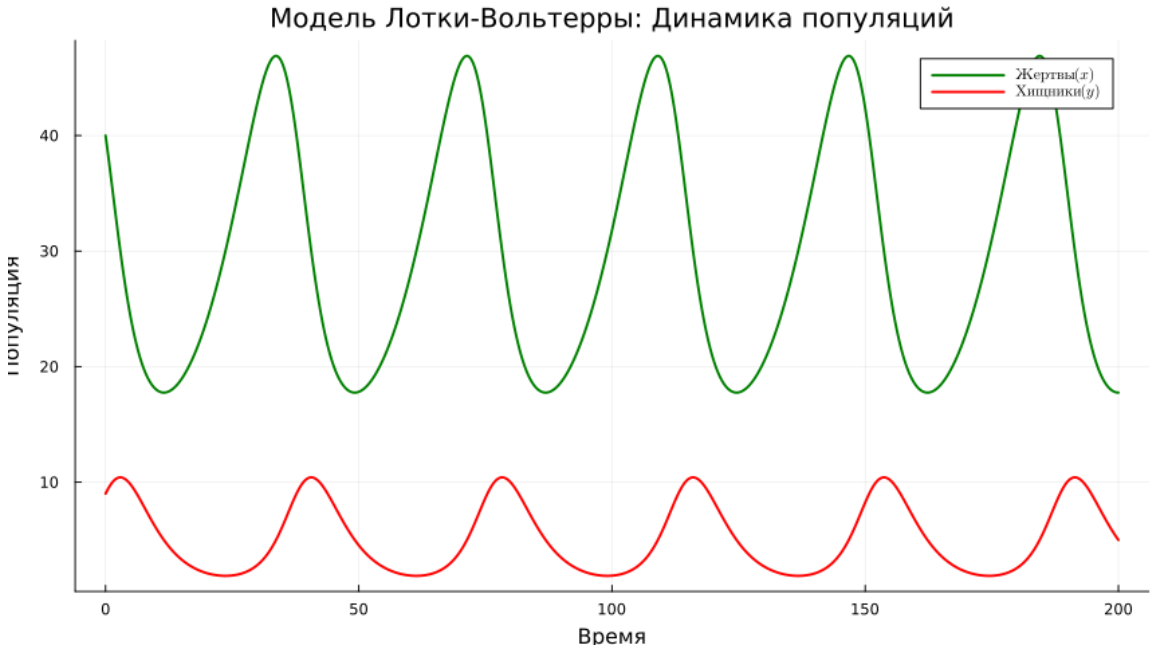
========== 8. ВИЗУАЛИЗАЦИЯ БАЗОВОГО ЭКСПЕРИМЕНТА ==========

6.8 8.1 График 1: Динамика популяций во времени

Отображает изменение численности жертв и хищников с течением времени.

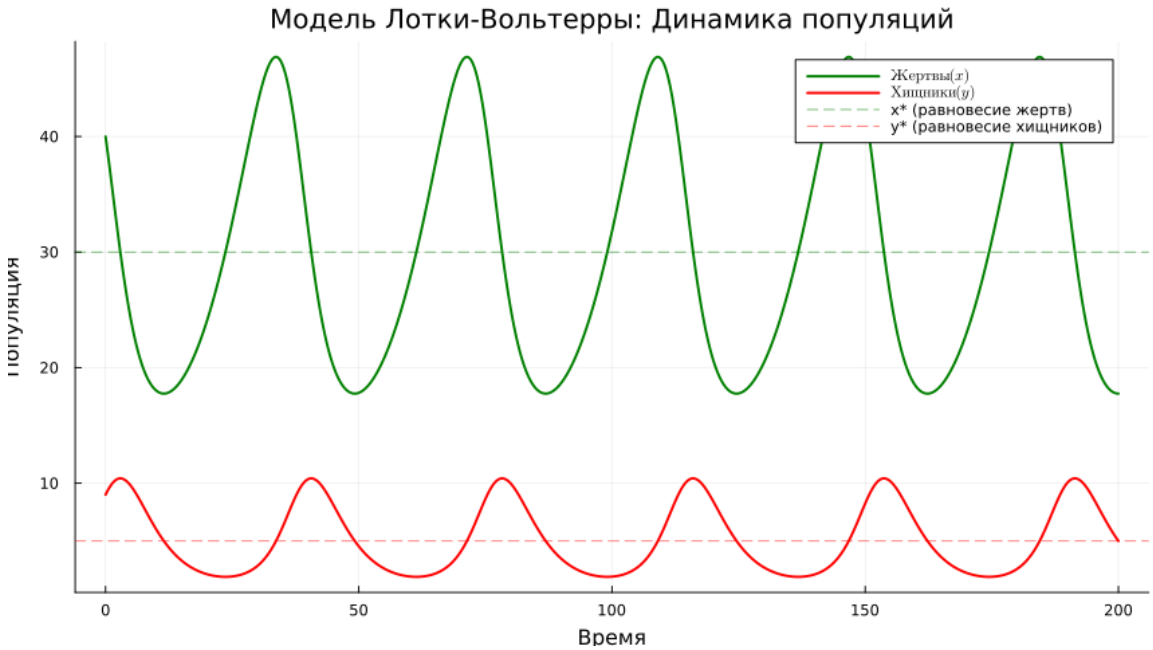
```
plt1 = plot(df_lv.t, [df_lv.prey df_lv.predator],
    label=[L"Жертвы (x)" L"Хищники (y)"],
    xlabel="Время",
    ylabel="Популяция",
    title="Модель Лотки-Вольтерры: Динамика популяций",
```

```
    linewidth=2,
    legend=:topright,
    grid=true,
    size=(900, 500),
    color=[:green :red])
```



Добавление стационарных уровней

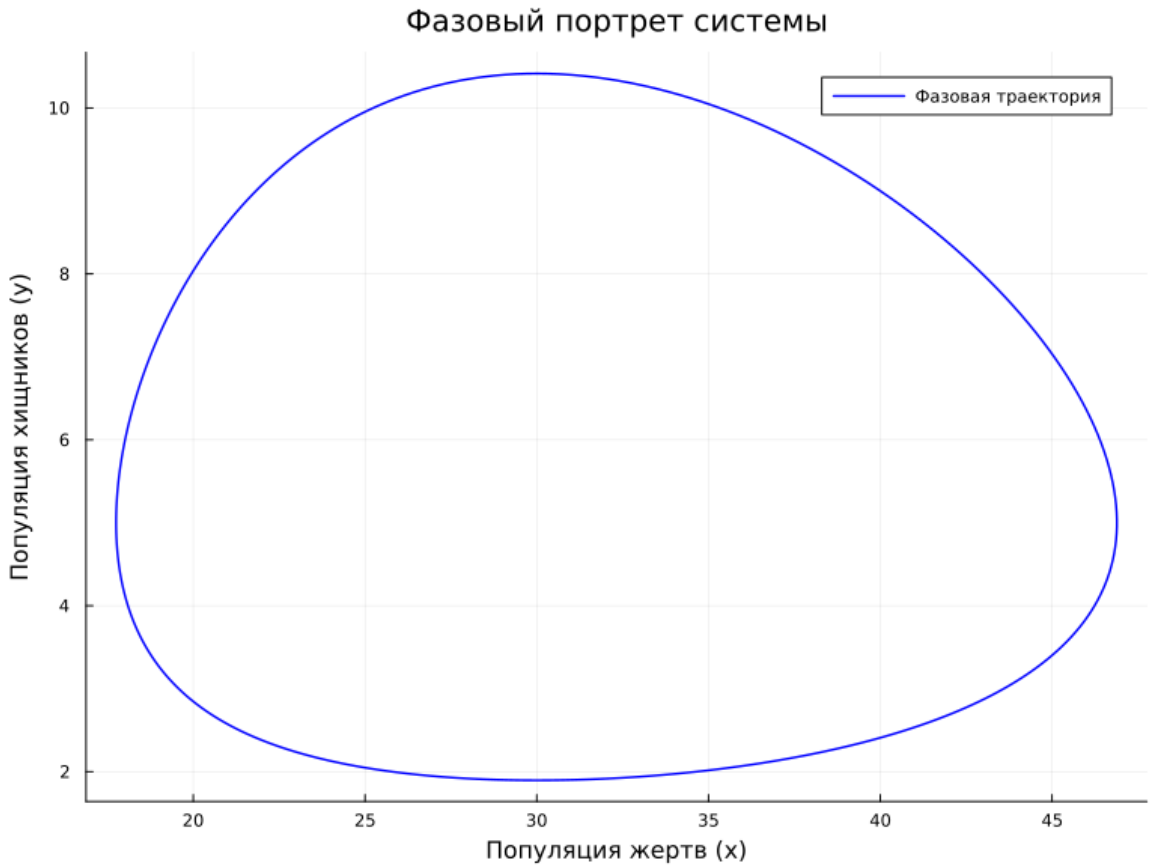
```
hline!(plt1, [x_star], color=:green, linestyle=:dash, alpha=0.5,
       label="x* (равновесие жертв)")
hline!(plt1, [y_star], color=:red, linestyle=:dash, alpha=0.5,
       label="y* (равновесие хищников)")
```



6.9 8.2 График 2: Фазовый портрет (хищники vs жертвы)

Показывает зависимость численности хищников от численности жертв. Замкнутые траектории указывают на циклический характер системы.

```
plt2 = plot(df_lv.prey, df_lv.predator,
    label="Фазовая траектория",
    xlabel="Популяция жертв (x)",
    ylabel="Популяция хищников (y)",
    title="Фазовый портрет системы",
    color=:blue,
    linewidth=1.5,
    grid=true,
    size=(800, 600),
    legend=:topright)
```

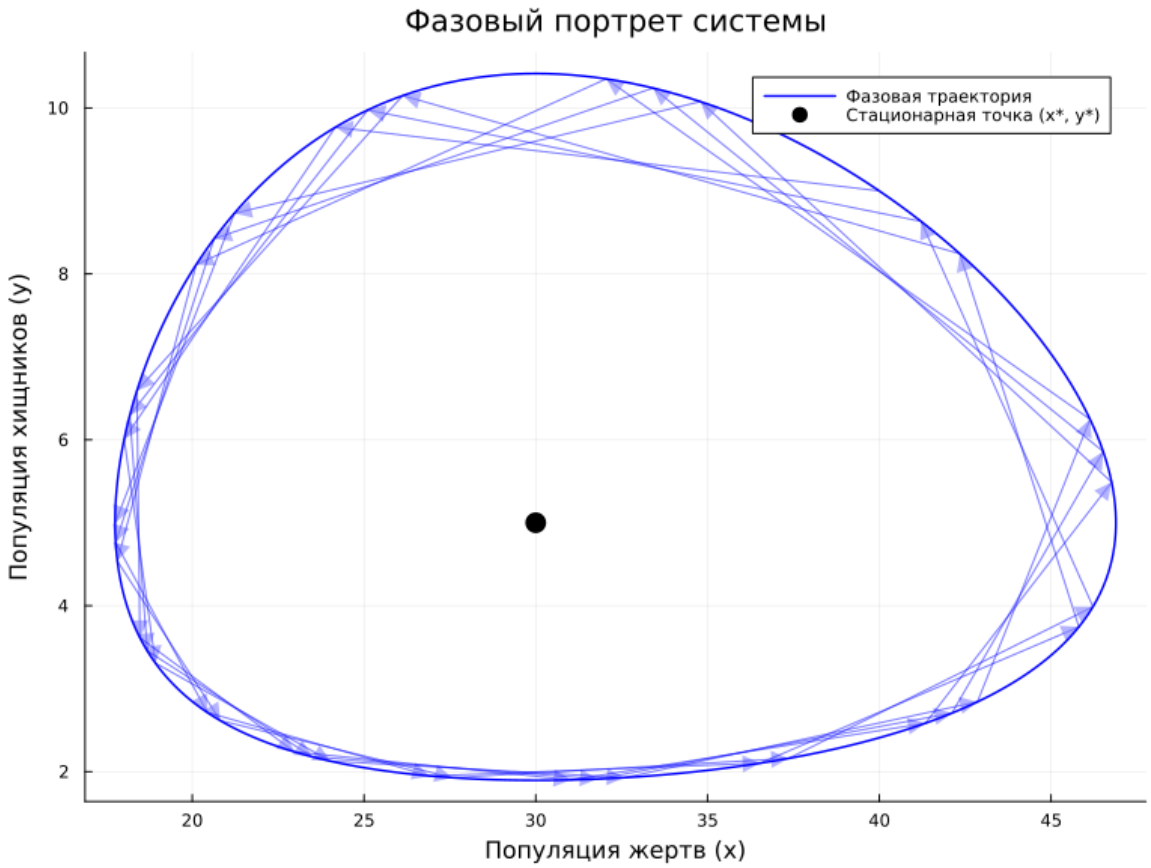


Добавление стрелок направления на фазовом портрете

```
step = 50
for i in 1:step:length(df_lv.prey)-step
    plot!(plt2,
        [df_lv.prey[i], df_lv.prey[i+step]],
        [df_lv.predator[i], df_lv.predator[i+step]],
        arrow=:closed, color=:blue, alpha=0.3, label=false)
end
```

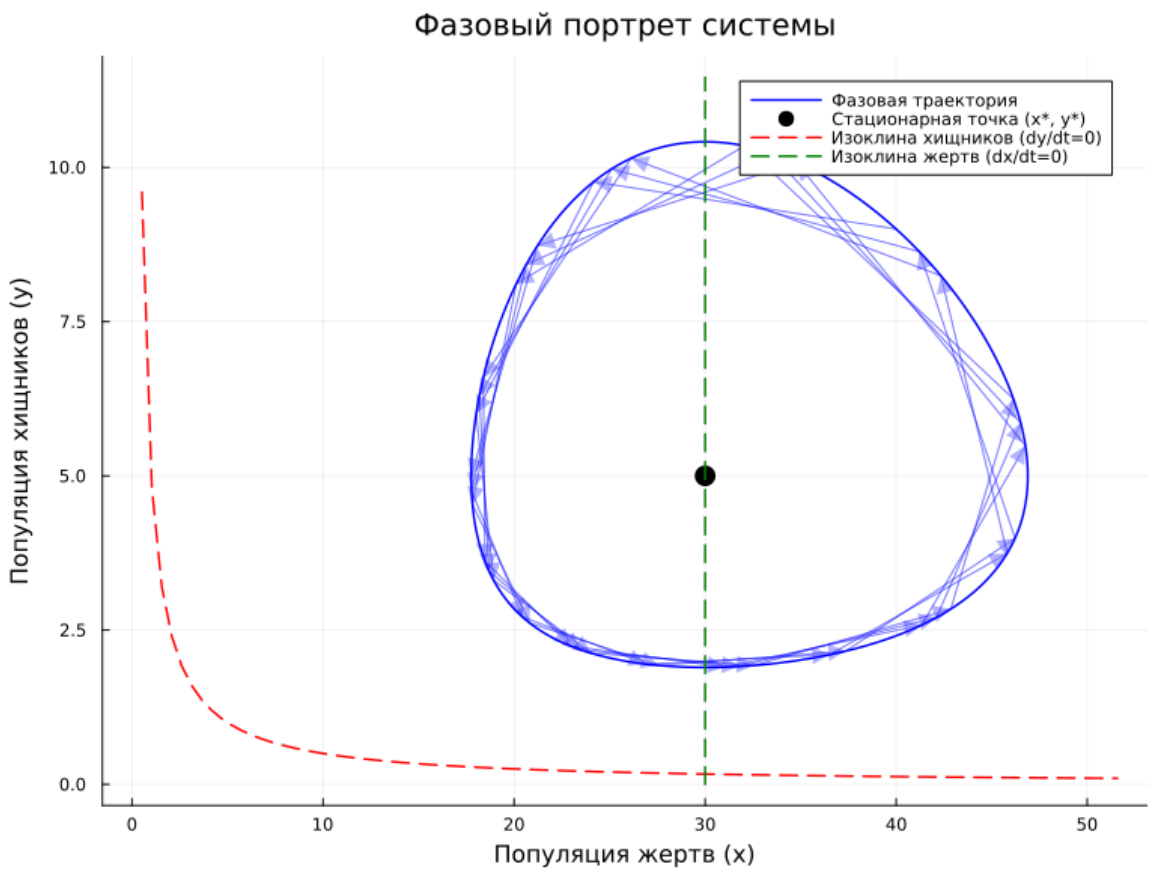
Добавление стационарной точки

```
scatter!(plt2, [x_star], [y_star],
    color=:black, markersize=8, label="Стационарная точка (x*, y*)")
```



Изоклины (линии нулевого роста)

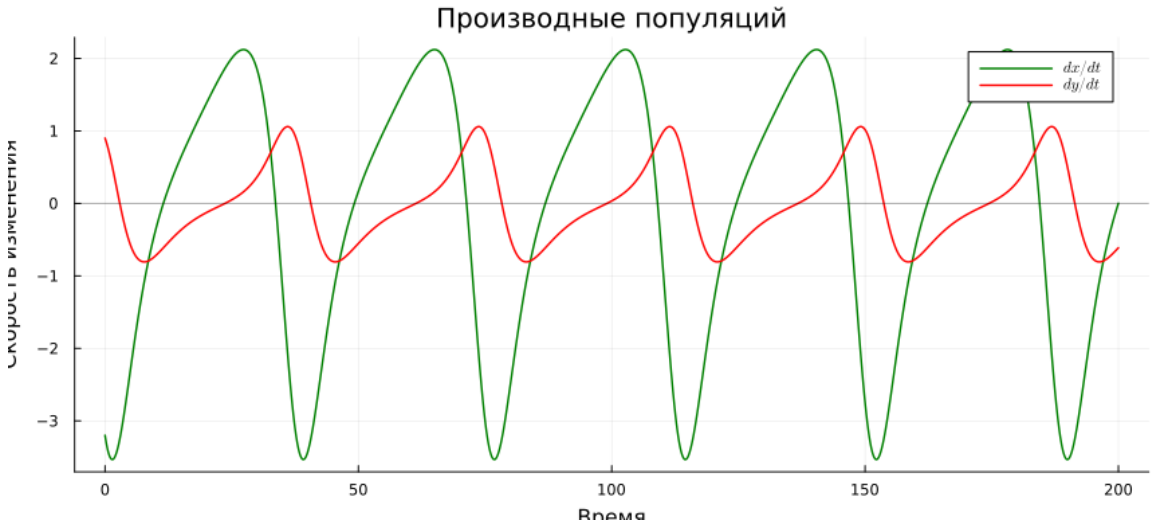
```
x_range = LinRange(0, maximum(df_lv.prey)*1.1, 100)
y_nullcline = p_lv[1] ./ (p_lv[2] .* x_range)  # dy/dt = 0
plot!(plt2, x_range, y_nullcline,
    color=:red, linestyle=:dash, linewidth=1.5, label="Изоклина хищников (dy/dt=0)")

y_range = LinRange(0, maximum(df_lv.predator)*1.1, 100)
x_nullcline = p_lv[4] ./ (p_lv[3] .* ones(length(y_range)))  # dx/dt = 0
plot!(plt2, x_nullcline, y_range,
    color=:green, linestyle=:dash, linewidth=1.5, label="Изоклина жертв (dx/dt=0)")
```



6.10 8.3 График 3: Производные (скорости изменения)

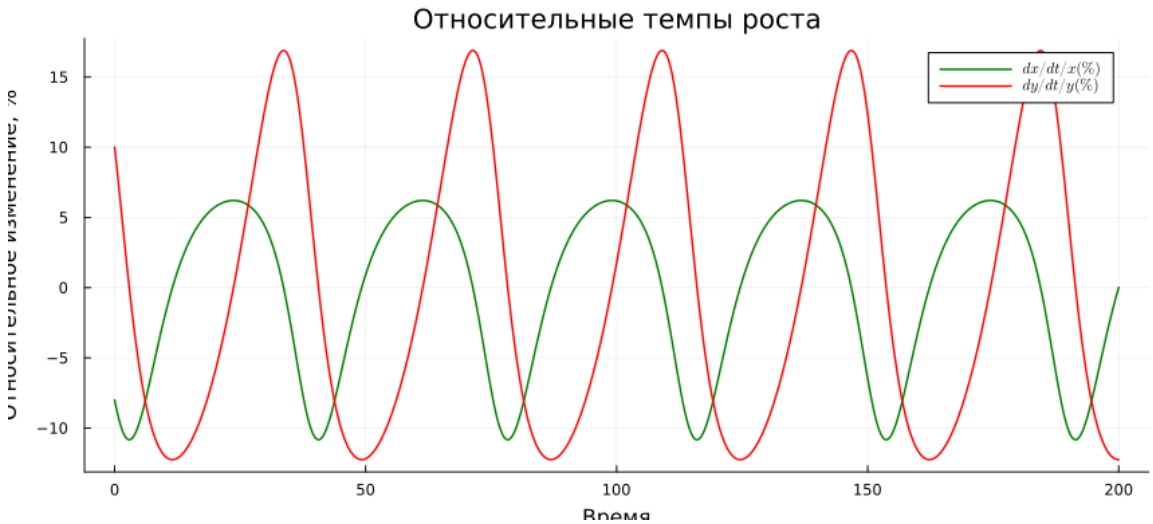
```
plt3 = plot(df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
    label=[L"dx/dt" L"dy/dt"],
    xlabel="Время",
    ylabel="Скорость изменения",
    title="Производные популяций",
    linewidth=1.5,
    legend=:topright,
    grid=true,
```

```
    size=(900, 400),
    color=[:green :red])

hline!(plt3, [0], color=:black, linestyle=:solid, alpha=0.3, label=false)
```



6.11 8.4 График 4: Относительные изменения (в процентах)

```
plt4 = plot(df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_change],
    label=[L"dx/dt / x (\%)" L"dy/dt / y (\%)"],
    xlabel="Время",
    ylabel="Относительное изменение, %",
    title="Относительные темпы роста",
    linewidth=1.5,
    legend=:topright,
    grid=true,
    size=(900, 400),
    color=[:green :red])
```



6.12 8.5 График 5: Спектральный анализ (FFT)

Функция для вычисления быстрого преобразования Фурье

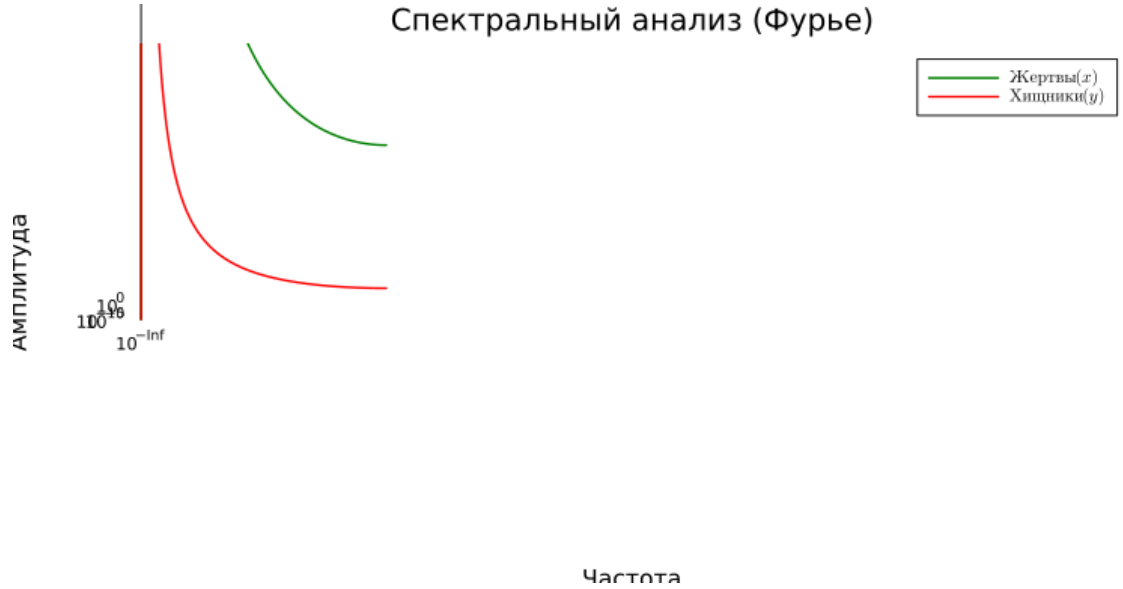
```
function compute_fft(signal, dt)
    n = length(signal)
    spectrum = abs.(rfft(signal .- mean(signal)))
    freq = rfftfreq(n, 1/dt)
    return freq, spectrum
end
```

```
compute_fft (generic function with 1 method)
```

Вычисление периодов колебаний

```
freq_prey, spectrum_prey = compute_fft(df_lv.prey, dt_lv)
freq_predator, spectrum_predator = compute_fft(df_lv.predator, dt_lv)

plt5 = plot(freq_prey, [spectrum_prey spectrum_predator],
    label=[L"Жертвы (x)" L"Хищники (y)"],
```

```
    xlabel="Частота",
    ylabel="Амплитуда",
    title="Спектральный анализ (Фурье)",
    linewidth=1.5,
    xscale=:log10,
    yscale=:log10,
    legend=:topright,
    grid=true,
    size=(800, 400),
    color=[:green :red])
```




```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xsca
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xsca
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
```

Нахождение доминирующих частот

```
if length(spectrum_prey) > 0
    idx_prey = argmax(spectrum_prey[2:end]) + 1  # пропускаем нулевую частоту
    dominant_freq_prey = freq_prey[idx_prey]
    period_prey = 1/dominant_freq_prey

    println("\nСпектральный анализ:")
    println("  Доминирующая частота колебаний жертв: ", round(dominant_freq_prey, dig
    println("  Период колебаний жертв: ", round(period_prey, digits=2), " ед. времени
end
```

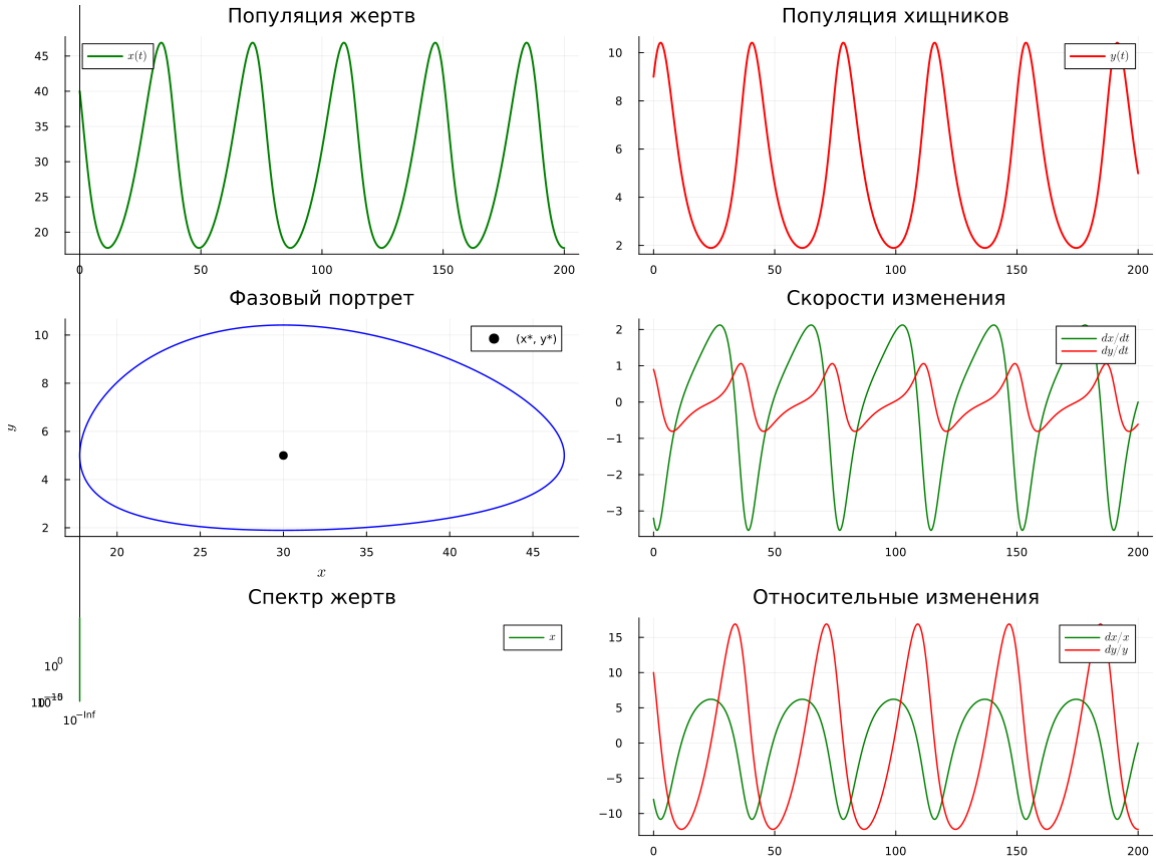
```
Спектральный анализ:
  Доминирующая частота колебаний жертв: 0.2499
  Период колебаний жертв: 4.0 ед. времени
```

6.13 8.6 График 6: Компактная панель всех графиков

```
plt6 = plot(layout=(3, 2), size=(1200, 900))

plot!(plt6[1], df_lv.t, df_lv.prey, label=L"x(t)", color=:green, linewidth=2,
    title="Популяция жертв", grid=true)
plot!(plt6[2], df_lv.t, df_lv.predator, label=L"y(t)", color=:red, linewidth=2,
    title="Популяция хищников", grid=true)
plot!(plt6[3], df_lv.prey, df_lv.predator, label=false, color=:blue, linewidth=1.5,
    title="Фазовый портрет", xlabel=L"x", ylabel=L"y", grid=true)
scatter!(plt6[3], [x_star], [y_star], color=:black, markersize=5, label="(x*, y*)")
plot!(plt6[4], df_lv.t, [df_lv.dprey_dt df_lv.dpredator_dt],
```

```
    label=[L"dx/dt" L"dy/dt"], color=[:green :red], linewidth=1.5,
    title="Скорости изменения", grid=true, legend=:topright)
plot!(plt6[5], freq_prey, spectrum_prey, label=L"x", color=:green, linewidth=1.5,
    title="Спектр жертв", xscale=:log10, yscale=:log10, grid=true)
plot!(plt6[6], df_lv.t, [df_lv.prey_pct_change df_lv.predator_pct_change],
    label=[L"dx/x" L"dy/y"], color=[:green :red], linewidth=1.5,
    title="Относительные изменения", grid=true, legend=:topright)
```

```
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xsca
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
```



========== 9. АНАЛИЗ БАЗОВОГО ЭКСПЕРИМЕНТА ==========

```
println("\n" * "="^60)
println("АНАЛИЗ БАЗОВОГО ЭКСПЕРИМЕНТА")
println("="^60)

println("\nОсновные статистики:")
println("  Жертвы: min = $(round(minimum(df_lv.prey), digits=2)), " *
        "max = $(round(maximum(df_lv.prey), digits=2)), " *
        "mean = $(round(mean(df_lv.prey), digits=2))")
println("  Хищники: min = $(round(minimum(df_lv.predator), digits=2)), " *
        "max = $(round(maximum(df_lv.predator), digits=2)), " *
```

```
        "mean = $(round(mean(df_lv.predator), digits=2))")
```

```
============================================================
АНАЛИЗ БАЗОВОГО ЭКСПЕРИМЕНТА
============================================================

Основные статистики:
  Жертвы: min = 17.75, max = 46.89, mean = 29.71
  Хищники: min = 1.9, max = 10.41, mean = 5.2
```

6.14 9.1 Анализ колебаний

Функция для поиска первого пика в сигнале

```
function find_first_peak(signal, time)
    for i in 2:length(signal)-1
        if signal[i] > signal[i-1] && signal[i] > signal[i+1]
            return time[i], signal[i]
        end
    end
    return NaN, NaN
end

peak_time_prey, peak_value_prey = find_first_peak(df_lv.prey, df_lv.t)
peak_time_predator, peak_value_predator = find_first_peak(df_lv.predator, df_lv.t)

if !isnan(peak_time_prey) && !isnan(peak_time_predator)
    phase_shift = peak_time_predator - peak_time_prey
```

```
    println("\nАнализ колебаний:")
    println("  Первый пик жертв: время = $(round(peak_time_prey, digits=2)), " *
            "значение = $(round(peak_value_prey, digits=2))")
    println("  Первый пик хищников: время = $(round(peak_time_predator, digits=2)), "
            "значение = $(round(peak_value_predator, digits=2))")
    println("  Сдвиг фаз (хищники отстают): $(round(phase_shift, digits=2)) ед. времe
end
```

```
Анализ колебаний:
  Первый пик жертв: время = 33.7, значение = 46.89
  Первый пик хищников: время = 2.9, значение = 10.41
  Сдвиг фаз (хищники отстают): -30.8 ед. времени

  ============================================================ 10.
ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ ==========================================
```

6.15 10.1 Зачем нужно параметрическое сканирование?

До сих пор мы рассматривали только один набор параметров. Но как изменится динамика системы, если: - **α** (скорость размножения жертв) будет другой? - **β** (скорость поедания) изменится? - **δ** (эффективность конверсии) будет иной? - **γ** (смертность хищников) поменяется?

Чтобы ответить на эти вопросы, проведём **параметрическое сканирование** - запустим модель с разными комбинациями параметров и сравним результаты.

```
println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("="^60)
```

```
============================================================
ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ
============================================================
```

6.16 10.2 Сетка параметров

Создадим словарь с наборами значений для каждого параметра. Мы будем варьировать 4 параметра: - **α**: скорость размножения жертв - **β**: скорость поедания жертв - **δ**: коэффициент конверсии - **γ**: смертность хищников

```
param_grid = Dict(
    :α => [0.05, 0.1, 0.15, 0.2],        # скорость размножения жертв
    :β => [0.01, 0.02, 0.03, 0.04],       # скорость поедания жертв
    :δ => [0.005, 0.01, 0.015, 0.02],     # коэффициент конверсии
    :γ => [0.2, 0.3, 0.4, 0.5],           # смертность хищников
    :u0 => [[40.0, 9.0]],                  # начальные условия (фиксированы)
    :tspan => [(0.0, 200.0)]               # время моделирования (фиксировано)
)
```

```
Dict{Symbol, Vector} with 6 entries:
  :γ     => [0.05, 0.1, 0.15, 0.2]
  :β     => [0.2, 0.3, 0.4, 0.5]
  :u0    => [[40.0, 9.0]]
  :δ     => [0.005, 0.01, 0.015, 0.02]
```

```
:□     => [0.01, 0.02, 0.03, 0.04]
:tspan => [(0.0, 200.0)]
```

Создаём все возможные комбинации параметров вручную Всего комбинаций: 4 × 4 × 4 × 4 = 256 экспериментов

```
println("\nСоздание комбинаций параметров...")

all_params = []
for □ in param_grid[:□]
    for □ in param_grid[:□]
        for □ in param_grid[:□]
            for □ in param_grid[:□]
                push!(all_params, Dict(
                    :□ => □,
                    :□ => □,
                    :□ => □,
                    :□ => □,
                    :u0 => param_grid[:u0][1],
                    :tspan => param_grid[:tspan][1]
                ))
            end
        end
    end
end

println("Всего комбинаций параметров: ", length(all_params))  # 4×4×4×4 = 256
println("\nИсследуемые значения:")
println("  □: ", param_grid[:□])
println("  □: ", param_grid[:□])
```

```
println("  □: ", param_grid[:□])
println("  □: ", param_grid[:□])
```

```
Создание комбинаций параметров...
Всего комбинаций параметров: 256

Исследуемые значения:
  □: [0.05, 0.1, 0.15, 0.2]
  □: [0.01, 0.02, 0.03, 0.04]
  □: [0.005, 0.01, 0.015, 0.02]
  □: [0.2, 0.3, 0.4, 0.5]
```

6.17 10.3 Функция для запуска одного эксперимента

Эта функция принимает набор параметров и возвращает результаты. Она будет вызываться для каждой комбинации параметров.

```
function run_lv_experiment(params::Dict)
    □ = params[:□]# Извлекаем параметры из словаря
    □ = params[:□]
    □ = params[:□]
    □ = params[:□]
    u0 = params[:u0]
    tspan = params[:tspan]

    p = [□, □, □, □]
    prob = ODEProblem(lotka_volterra!, u0, tspan, p) # Создаём и решаем задачу
    sol = solve(prob, Tsit5(), saveat=1.0)  # сохраняем реже для экономии памяти
```

```
    df_temp = DataFrame(t=sol.t) # Анализируем результаты
    df_temp.prey = [u[1] for u in sol.u]
    df_temp.predator = [u[2] for u in sol.u]
    x_star_val = □ / □ # Стационарные точки
    y_star_val = □ / □

    mean_prey = mean(df_temp.prey) # Статистика
    mean_predator = mean(df_temp.predator)
    min_prey = minimum(df_temp.prey)
    max_prey = maximum(df_temp.prey)
    min_predator = minimum(df_temp.predator)
    max_predator = maximum(df_temp.predator)

    prey_amplitude = (max_prey - min_prey) / 2 # Амплитуда колебаний
    predator_amplitude = (max_predator - min_predator) / 2

    return Dict(
        "solution" => sol,
        "mean_prey" => mean_prey,
        "mean_predator" => mean_predator,
        "min_prey" => min_prey,
        "max_prey" => max_prey,
        "min_predator" => min_predator,
        "max_predator" => max_predator,
        "prey_amplitude" => prey_amplitude,
        "predator_amplitude" => predator_amplitude,
        "x_star" => x_star_val,
        "y_star" => y_star_val
```

```
    )
end
```

```
run_lv_experiment (generic function with 1 method)
```

6.18 10.4 Запуск всех экспериментов

Используем функцию produce_or_load для кэширования результатов. Это позволяет не пересчитывать эксперименты при повторных запусках.

```
all_results = []

for (i, params) in enumerate(all_params)
    println("\nПрогресс: $i/$(length(all_params))")
    println("  □ = $(params[:□]), □ = $(params[:□]), □ = $(params[:□]), □ = $(params[

    data, path = produce_or_load( # Запускаем эксперимент (или загружаем из кэша)
        datadir(script_name, "parametric_scan"),
        params,
        run_lv_experiment,
        prefix = "lv_scan",
        tag = false,
        verbose = false
    )

    result_summary = merge(params, Dict( # Сохраняем сводку результатов
        :mean_prey => data["mean_prey"],
        :mean_predator => data["mean_predator"],
        :prey_amplitude => data["prey_amplitude"],
```

```
        :predator_amplitude => data["predator_amplitude"],
        :x_star => data["x_star"],
        :y_star => data["y_star"]
    ))

    push!(all_results, result_summary)
end
```

```
Прогресс: 1/256
  □ = 0.05, □ = 0.01, □ = 0.005, □ = 0.2

Прогресс: 2/256
  □ = 0.05, □ = 0.01, □ = 0.005, □ = 0.3

Прогресс: 3/256
  □ = 0.05, □ = 0.01, □ = 0.005, □ = 0.4

Прогресс: 4/256
  □ = 0.05, □ = 0.01, □ = 0.005, □ = 0.5

Прогресс: 5/256
  □ = 0.05, □ = 0.01, □ = 0.01, □ = 0.2

Прогресс: 6/256
  □ = 0.05, □ = 0.01, □ = 0.01, □ = 0.3

Прогресс: 7/256
  □ = 0.05, □ = 0.01, □ = 0.01, □ = 0.4
```

Прогресс: 8/256

  □ = 0.05, □ = 0.01, □ = 0.01, □ = 0.5

Прогресс: 9/256

  □ = 0.05, □ = 0.01, □ = 0.015, □ = 0.2

Прогресс: 10/256

  □ = 0.05, □ = 0.01, □ = 0.015, □ = 0.3

Прогресс: 11/256

  □ = 0.05, □ = 0.01, □ = 0.015, □ = 0.4

Прогресс: 12/256

  □ = 0.05, □ = 0.01, □ = 0.015, □ = 0.5

Прогресс: 13/256

  □ = 0.05, □ = 0.01, □ = 0.02, □ = 0.2

Прогресс: 14/256

  □ = 0.05, □ = 0.01, □ = 0.02, □ = 0.3

Прогресс: 15/256

  □ = 0.05, □ = 0.01, □ = 0.02, □ = 0.4

Прогресс: 16/256

  □ = 0.05, □ = 0.01, □ = 0.02, □ = 0.5

Прогресс: 17/256

□ = 0.05, □ = 0.02, □ = 0.005, □ = 0.2

Прогресс: 18/256

□ = 0.05, □ = 0.02, □ = 0.005, □ = 0.3

Прогресс: 19/256

□ = 0.05, □ = 0.02, □ = 0.005, □ = 0.4

Прогресс: 20/256

□ = 0.05, □ = 0.02, □ = 0.005, □ = 0.5

Прогресс: 21/256

□ = 0.05, □ = 0.02, □ = 0.01, □ = 0.2

Прогресс: 22/256

□ = 0.05, □ = 0.02, □ = 0.01, □ = 0.3

Прогресс: 23/256

□ = 0.05, □ = 0.02, □ = 0.01, □ = 0.4

Прогресс: 24/256

□ = 0.05, □ = 0.02, □ = 0.01, □ = 0.5

Прогресс: 25/256

□ = 0.05, □ = 0.02, □ = 0.015, □ = 0.2

Прогресс: 26/256

□ = 0.05, □ = 0.02, □ = 0.015, □ = 0.3

Прогресс: 27/256

  = 0.05, = 0.02, = 0.015, = 0.4

Прогресс: 28/256

  = 0.05, = 0.02, = 0.015, = 0.5

Прогресс: 29/256

  = 0.05, = 0.02, = 0.02, = 0.2

Прогресс: 30/256

  = 0.05, = 0.02, = 0.02, = 0.3

Прогресс: 31/256

  = 0.05, = 0.02, = 0.02, = 0.4

Прогресс: 32/256

  = 0.05, = 0.02, = 0.02, = 0.5

Прогресс: 33/256

  = 0.05, = 0.03, = 0.005, = 0.2

Прогресс: 34/256

  = 0.05, = 0.03, = 0.005, = 0.3

Прогресс: 35/256

  = 0.05, = 0.03, = 0.005, = 0.4

Прогресс: 36/256

  = 0.05, = 0.03, = 0.005, = 0.5

Прогресс: 37/256

  □ = 0.05, □ = 0.03, □ = 0.01, □ = 0.2

Прогресс: 38/256

  □ = 0.05, □ = 0.03, □ = 0.01, □ = 0.3

Прогресс: 39/256

  □ = 0.05, □ = 0.03, □ = 0.01, □ = 0.4

Прогресс: 40/256

  □ = 0.05, □ = 0.03, □ = 0.01, □ = 0.5

Прогресс: 41/256

  □ = 0.05, □ = 0.03, □ = 0.015, □ = 0.2

Прогресс: 42/256

  □ = 0.05, □ = 0.03, □ = 0.015, □ = 0.3

Прогресс: 43/256

  □ = 0.05, □ = 0.03, □ = 0.015, □ = 0.4

Прогресс: 44/256

  □ = 0.05, □ = 0.03, □ = 0.015, □ = 0.5

Прогресс: 45/256

  □ = 0.05, □ = 0.03, □ = 0.02, □ = 0.2

Прогресс: 46/256

□ = 0.05, □ = 0.03, □ = 0.02, □ = 0.3

Прогресс: 47/256

□ = 0.05, □ = 0.03, □ = 0.02, □ = 0.4

Прогресс: 48/256

□ = 0.05, □ = 0.03, □ = 0.02, □ = 0.5

Прогресс: 49/256

□ = 0.05, □ = 0.04, □ = 0.005, □ = 0.2

Прогресс: 50/256

□ = 0.05, □ = 0.04, □ = 0.005, □ = 0.3

Прогресс: 51/256

□ = 0.05, □ = 0.04, □ = 0.005, □ = 0.4

Прогресс: 52/256

□ = 0.05, □ = 0.04, □ = 0.005, □ = 0.5

Прогресс: 53/256

□ = 0.05, □ = 0.04, □ = 0.01, □ = 0.2

Прогресс: 54/256

□ = 0.05, □ = 0.04, □ = 0.01, □ = 0.3

Прогресс: 55/256

□ = 0.05, □ = 0.04, □ = 0.01, □ = 0.4

Прогресс: 56/256

  = 0.05, = 0.04, = 0.01, = 0.5

Прогресс: 57/256

  = 0.05, = 0.04, = 0.015, = 0.2

Прогресс: 58/256

  = 0.05, = 0.04, = 0.015, = 0.3

Прогресс: 59/256

  = 0.05, = 0.04, = 0.015, = 0.4

Прогресс: 60/256

  = 0.05, = 0.04, = 0.015, = 0.5

Прогресс: 61/256

  = 0.05, = 0.04, = 0.02, = 0.2

Прогресс: 62/256

  = 0.05, = 0.04, = 0.02, = 0.3

Прогресс: 63/256

  = 0.05, = 0.04, = 0.02, = 0.4

Прогресс: 64/256

  = 0.05, = 0.04, = 0.02, = 0.5

Прогресс: 65/256

  = 0.1, = 0.01, = 0.005, = 0.2

Прогресс: 66/256

□ = 0.1, □ = 0.01, □ = 0.005, □ = 0.3

Прогресс: 67/256

□ = 0.1, □ = 0.01, □ = 0.005, □ = 0.4

Прогресс: 68/256

□ = 0.1, □ = 0.01, □ = 0.005, □ = 0.5

Прогресс: 69/256

□ = 0.1, □ = 0.01, □ = 0.01, □ = 0.2

Прогресс: 70/256

□ = 0.1, □ = 0.01, □ = 0.01, □ = 0.3

Прогресс: 71/256

□ = 0.1, □ = 0.01, □ = 0.01, □ = 0.4

Прогресс: 72/256

□ = 0.1, □ = 0.01, □ = 0.01, □ = 0.5

Прогресс: 73/256

□ = 0.1, □ = 0.01, □ = 0.015, □ = 0.2

Прогресс: 74/256

□ = 0.1, □ = 0.01, □ = 0.015, □ = 0.3

Прогресс: 75/256

□ = 0.1, □ = 0.01, □ = 0.015, □ = 0.4

Прогресс: 76/256

□ = 0.1, □ = 0.01, □ = 0.015, □ = 0.5

Прогресс: 77/256

□ = 0.1, □ = 0.01, □ = 0.02, □ = 0.2

Прогресс: 78/256

□ = 0.1, □ = 0.01, □ = 0.02, □ = 0.3

Прогресс: 79/256

□ = 0.1, □ = 0.01, □ = 0.02, □ = 0.4

Прогресс: 80/256

□ = 0.1, □ = 0.01, □ = 0.02, □ = 0.5

Прогресс: 81/256

□ = 0.1, □ = 0.02, □ = 0.005, □ = 0.2

Прогресс: 82/256

□ = 0.1, □ = 0.02, □ = 0.005, □ = 0.3

Прогресс: 83/256

□ = 0.1, □ = 0.02, □ = 0.005, □ = 0.4

Прогресс: 84/256

□ = 0.1, □ = 0.02, □ = 0.005, □ = 0.5

Прогресс: 85/256

□ = 0.1, □ = 0.02, □ = 0.01, □ = 0.2

Прогресс: 86/256

□ = 0.1, □ = 0.02, □ = 0.01, □ = 0.3

Прогресс: 87/256

□ = 0.1, □ = 0.02, □ = 0.01, □ = 0.4

Прогресс: 88/256

□ = 0.1, □ = 0.02, □ = 0.01, □ = 0.5

Прогресс: 89/256

□ = 0.1, □ = 0.02, □ = 0.015, □ = 0.2

Прогресс: 90/256

□ = 0.1, □ = 0.02, □ = 0.015, □ = 0.3

Прогресс: 91/256

□ = 0.1, □ = 0.02, □ = 0.015, □ = 0.4

Прогресс: 92/256

□ = 0.1, □ = 0.02, □ = 0.015, □ = 0.5

Прогресс: 93/256

□ = 0.1, □ = 0.02, □ = 0.02, □ = 0.2

Прогресс: 94/256

□ = 0.1, □ = 0.02, □ = 0.02, □ = 0.3

Прогресс: 95/256

□ = 0.1, □ = 0.02, □ = 0.02, □ = 0.4

Прогресс: 96/256

□ = 0.1, □ = 0.02, □ = 0.02, □ = 0.5

Прогресс: 97/256

□ = 0.1, □ = 0.03, □ = 0.005, □ = 0.2

Прогресс: 98/256

□ = 0.1, □ = 0.03, □ = 0.005, □ = 0.3

Прогресс: 99/256

□ = 0.1, □ = 0.03, □ = 0.005, □ = 0.4

Прогресс: 100/256

□ = 0.1, □ = 0.03, □ = 0.005, □ = 0.5

Прогресс: 101/256

□ = 0.1, □ = 0.03, □ = 0.01, □ = 0.2

Прогресс: 102/256

□ = 0.1, □ = 0.03, □ = 0.01, □ = 0.3

Прогресс: 103/256

□ = 0.1, □ = 0.03, □ = 0.01, □ = 0.4

Прогресс: 104/256

□ = 0.1, □ = 0.03, □ = 0.01, □ = 0.5

Прогресс: 105/256

□ = 0.1, □ = 0.03, □ = 0.015, □ = 0.2

Прогресс: 106/256

□ = 0.1, □ = 0.03, □ = 0.015, □ = 0.3

Прогресс: 107/256

□ = 0.1, □ = 0.03, □ = 0.015, □ = 0.4

Прогресс: 108/256

□ = 0.1, □ = 0.03, □ = 0.015, □ = 0.5

Прогресс: 109/256

□ = 0.1, □ = 0.03, □ = 0.02, □ = 0.2

Прогресс: 110/256

□ = 0.1, □ = 0.03, □ = 0.02, □ = 0.3

Прогресс: 111/256

□ = 0.1, □ = 0.03, □ = 0.02, □ = 0.4

Прогресс: 112/256

□ = 0.1, □ = 0.03, □ = 0.02, □ = 0.5

Прогресс: 113/256

□ = 0.1, □ = 0.04, □ = 0.005, □ = 0.2

Прогресс: 114/256

  = 0.1, = 0.04, = 0.005, = 0.3

Прогресс: 115/256

  = 0.1, = 0.04, = 0.005, = 0.4

Прогресс: 116/256

  = 0.1, = 0.04, = 0.005, = 0.5

Прогресс: 117/256

  = 0.1, = 0.04, = 0.01, = 0.2

Прогресс: 118/256

  = 0.1, = 0.04, = 0.01, = 0.3

Прогресс: 119/256

  = 0.1, = 0.04, = 0.01, = 0.4

Прогресс: 120/256

  = 0.1, = 0.04, = 0.01, = 0.5

Прогресс: 121/256

  = 0.1, = 0.04, = 0.015, = 0.2

Прогресс: 122/256

  = 0.1, = 0.04, = 0.015, = 0.3

Прогресс: 123/256

  = 0.1, = 0.04, = 0.015, = 0.4

Прогресс: 124/256

□ = 0.1, □ = 0.04, □ = 0.015, □ = 0.5

Прогресс: 125/256

□ = 0.1, □ = 0.04, □ = 0.02, □ = 0.2

Прогресс: 126/256

□ = 0.1, □ = 0.04, □ = 0.02, □ = 0.3

Прогресс: 127/256

□ = 0.1, □ = 0.04, □ = 0.02, □ = 0.4

Прогресс: 128/256

□ = 0.1, □ = 0.04, □ = 0.02, □ = 0.5

Прогресс: 129/256

□ = 0.15, □ = 0.01, □ = 0.005, □ = 0.2

Прогресс: 130/256

□ = 0.15, □ = 0.01, □ = 0.005, □ = 0.3

Прогресс: 131/256

□ = 0.15, □ = 0.01, □ = 0.005, □ = 0.4

Прогресс: 132/256

□ = 0.15, □ = 0.01, □ = 0.005, □ = 0.5

Прогресс: 133/256

□ = 0.15, □ = 0.01, □ = 0.01, □ = 0.2

Прогресс: 134/256

□ = 0.15, □ = 0.01, □ = 0.01, □ = 0.3

Прогресс: 135/256

□ = 0.15, □ = 0.01, □ = 0.01, □ = 0.4

Прогресс: 136/256

□ = 0.15, □ = 0.01, □ = 0.01, □ = 0.5

Прогресс: 137/256

□ = 0.15, □ = 0.01, □ = 0.015, □ = 0.2

Прогресс: 138/256

□ = 0.15, □ = 0.01, □ = 0.015, □ = 0.3

Прогресс: 139/256

□ = 0.15, □ = 0.01, □ = 0.015, □ = 0.4

Прогресс: 140/256

□ = 0.15, □ = 0.01, □ = 0.015, □ = 0.5

Прогресс: 141/256

□ = 0.15, □ = 0.01, □ = 0.02, □ = 0.2

Прогресс: 142/256

□ = 0.15, □ = 0.01, □ = 0.02, □ = 0.3

Прогресс: 143/256

□ = 0.15, □ = 0.01, □ = 0.02, □ = 0.4

Прогресс: 144/256

□ = 0.15, □ = 0.01, □ = 0.02, □ = 0.5

Прогресс: 145/256

□ = 0.15, □ = 0.02, □ = 0.005, □ = 0.2

Прогресс: 146/256

□ = 0.15, □ = 0.02, □ = 0.005, □ = 0.3

Прогресс: 147/256

□ = 0.15, □ = 0.02, □ = 0.005, □ = 0.4

Прогресс: 148/256

□ = 0.15, □ = 0.02, □ = 0.005, □ = 0.5

Прогресс: 149/256

□ = 0.15, □ = 0.02, □ = 0.01, □ = 0.2

Прогресс: 150/256

□ = 0.15, □ = 0.02, □ = 0.01, □ = 0.3

Прогресс: 151/256

□ = 0.15, □ = 0.02, □ = 0.01, □ = 0.4

Прогресс: 152/256

□ = 0.15, □ = 0.02, □ = 0.01, □ = 0.5

Прогресс: 153/256

  □ = 0.15, □ = 0.02, □ = 0.015, □ = 0.2

Прогресс: 154/256

  □ = 0.15, □ = 0.02, □ = 0.015, □ = 0.3

Прогресс: 155/256

  □ = 0.15, □ = 0.02, □ = 0.015, □ = 0.4

Прогресс: 156/256

  □ = 0.15, □ = 0.02, □ = 0.015, □ = 0.5

Прогресс: 157/256

  □ = 0.15, □ = 0.02, □ = 0.02, □ = 0.2

Прогресс: 158/256

  □ = 0.15, □ = 0.02, □ = 0.02, □ = 0.3

Прогресс: 159/256

  □ = 0.15, □ = 0.02, □ = 0.02, □ = 0.4

Прогресс: 160/256

  □ = 0.15, □ = 0.02, □ = 0.02, □ = 0.5

Прогресс: 161/256

  □ = 0.15, □ = 0.03, □ = 0.005, □ = 0.2

Прогресс: 162/256

□ = 0.15, □ = 0.03, □ = 0.005, □ = 0.3

Прогресс: 163/256

□ = 0.15, □ = 0.03, □ = 0.005, □ = 0.4

Прогресс: 164/256

□ = 0.15, □ = 0.03, □ = 0.005, □ = 0.5

Прогресс: 165/256

□ = 0.15, □ = 0.03, □ = 0.01, □ = 0.2

Прогресс: 166/256

□ = 0.15, □ = 0.03, □ = 0.01, □ = 0.3

Прогресс: 167/256

□ = 0.15, □ = 0.03, □ = 0.01, □ = 0.4

Прогресс: 168/256

□ = 0.15, □ = 0.03, □ = 0.01, □ = 0.5

Прогресс: 169/256

□ = 0.15, □ = 0.03, □ = 0.015, □ = 0.2

Прогресс: 170/256

□ = 0.15, □ = 0.03, □ = 0.015, □ = 0.3

Прогресс: 171/256

□ = 0.15, □ = 0.03, □ = 0.015, □ = 0.4

Прогресс: 172/256

  = 0.15, = 0.03, = 0.015, = 0.5

Прогресс: 173/256

  = 0.15, = 0.03, = 0.02, = 0.2

Прогресс: 174/256

  = 0.15, = 0.03, = 0.02, = 0.3

Прогресс: 175/256

  = 0.15, = 0.03, = 0.02, = 0.4

Прогресс: 176/256

  = 0.15, = 0.03, = 0.02, = 0.5

Прогресс: 177/256

  = 0.15, = 0.04, = 0.005, = 0.2

Прогресс: 178/256

  = 0.15, = 0.04, = 0.005, = 0.3

Прогресс: 179/256

  = 0.15, = 0.04, = 0.005, = 0.4

Прогресс: 180/256

  = 0.15, = 0.04, = 0.005, = 0.5

Прогресс: 181/256

  = 0.15, = 0.04, = 0.01, = 0.2

Прогресс: 182/256

  = 0.15, = 0.04, = 0.01, = 0.3

Прогресс: 183/256

  = 0.15, = 0.04, = 0.01, = 0.4

Прогресс: 184/256

  = 0.15, = 0.04, = 0.01, = 0.5

Прогресс: 185/256

  = 0.15, = 0.04, = 0.015, = 0.2

Прогресс: 186/256

  = 0.15, = 0.04, = 0.015, = 0.3

Прогресс: 187/256

  = 0.15, = 0.04, = 0.015, = 0.4

Прогресс: 188/256

  = 0.15, = 0.04, = 0.015, = 0.5

Прогресс: 189/256

  = 0.15, = 0.04, = 0.02, = 0.2

Прогресс: 190/256

  = 0.15, = 0.04, = 0.02, = 0.3

Прогресс: 191/256

□ = 0.15, □ = 0.04, □ = 0.02, □ = 0.4

Прогресс: 192/256

□ = 0.15, □ = 0.04, □ = 0.02, □ = 0.5

Прогресс: 193/256

□ = 0.2, □ = 0.01, □ = 0.005, □ = 0.2

Прогресс: 194/256

□ = 0.2, □ = 0.01, □ = 0.005, □ = 0.3

Прогресс: 195/256

□ = 0.2, □ = 0.01, □ = 0.005, □ = 0.4

Прогресс: 196/256

□ = 0.2, □ = 0.01, □ = 0.005, □ = 0.5

Прогресс: 197/256

□ = 0.2, □ = 0.01, □ = 0.01, □ = 0.2

Прогресс: 198/256

□ = 0.2, □ = 0.01, □ = 0.01, □ = 0.3

Прогресс: 199/256

□ = 0.2, □ = 0.01, □ = 0.01, □ = 0.4

Прогресс: 200/256

□ = 0.2, □ = 0.01, □ = 0.01, □ = 0.5

Прогресс: 201/256

□ = 0.2, □ = 0.01, □ = 0.015, □ = 0.2

Прогресс: 202/256

□ = 0.2, □ = 0.01, □ = 0.015, □ = 0.3

Прогресс: 203/256

□ = 0.2, □ = 0.01, □ = 0.015, □ = 0.4

Прогресс: 204/256

□ = 0.2, □ = 0.01, □ = 0.015, □ = 0.5

Прогресс: 205/256

□ = 0.2, □ = 0.01, □ = 0.02, □ = 0.2

Прогресс: 206/256

□ = 0.2, □ = 0.01, □ = 0.02, □ = 0.3

Прогресс: 207/256

□ = 0.2, □ = 0.01, □ = 0.02, □ = 0.4

Прогресс: 208/256

□ = 0.2, □ = 0.01, □ = 0.02, □ = 0.5

Прогресс: 209/256

□ = 0.2, □ = 0.02, □ = 0.005, □ = 0.2

Прогресс: 210/256

□ = 0.2, □ = 0.02, □ = 0.005, □ = 0.3

Прогресс: 211/256

  = 0.2, = 0.02, = 0.005, = 0.4

Прогресс: 212/256

  = 0.2, = 0.02, = 0.005, = 0.5

Прогресс: 213/256

  = 0.2, = 0.02, = 0.01, = 0.2

Прогресс: 214/256

  = 0.2, = 0.02, = 0.01, = 0.3

Прогресс: 215/256

  = 0.2, = 0.02, = 0.01, = 0.4

Прогресс: 216/256

  = 0.2, = 0.02, = 0.01, = 0.5

Прогресс: 217/256

  = 0.2, = 0.02, = 0.015, = 0.2

Прогресс: 218/256

  = 0.2, = 0.02, = 0.015, = 0.3

Прогресс: 219/256

  = 0.2, = 0.02, = 0.015, = 0.4

Прогресс: 220/256

□ = 0.2, □ = 0.02, □ = 0.015, □ = 0.5

Прогресс: 221/256

□ = 0.2, □ = 0.02, □ = 0.02, □ = 0.2

Прогресс: 222/256

□ = 0.2, □ = 0.02, □ = 0.02, □ = 0.3

Прогресс: 223/256

□ = 0.2, □ = 0.02, □ = 0.02, □ = 0.4

Прогресс: 224/256

□ = 0.2, □ = 0.02, □ = 0.02, □ = 0.5

Прогресс: 225/256

□ = 0.2, □ = 0.03, □ = 0.005, □ = 0.2

Прогресс: 226/256

□ = 0.2, □ = 0.03, □ = 0.005, □ = 0.3

Прогресс: 227/256

□ = 0.2, □ = 0.03, □ = 0.005, □ = 0.4

Прогресс: 228/256

□ = 0.2, □ = 0.03, □ = 0.005, □ = 0.5

Прогресс: 229/256

□ = 0.2, □ = 0.03, □ = 0.01, □ = 0.2

Прогресс: 230/256

□ = 0.2, □ = 0.03, □ = 0.01, □ = 0.3

Прогресс: 231/256

□ = 0.2, □ = 0.03, □ = 0.01, □ = 0.4

Прогресс: 232/256

□ = 0.2, □ = 0.03, □ = 0.01, □ = 0.5

Прогресс: 233/256

□ = 0.2, □ = 0.03, □ = 0.015, □ = 0.2

Прогресс: 234/256

□ = 0.2, □ = 0.03, □ = 0.015, □ = 0.3

Прогресс: 235/256

□ = 0.2, □ = 0.03, □ = 0.015, □ = 0.4

Прогресс: 236/256

□ = 0.2, □ = 0.03, □ = 0.015, □ = 0.5

Прогресс: 237/256

□ = 0.2, □ = 0.03, □ = 0.02, □ = 0.2

Прогресс: 238/256

□ = 0.2, □ = 0.03, □ = 0.02, □ = 0.3

Прогресс: 239/256

□ = 0.2, □ = 0.03, □ = 0.02, □ = 0.4

Прогресс: 240/256

  = 0.2, = 0.03, = 0.02, = 0.5

Прогресс: 241/256

  = 0.2, = 0.04, = 0.005, = 0.2

Прогресс: 242/256

  = 0.2, = 0.04, = 0.005, = 0.3

Прогресс: 243/256

  = 0.2, = 0.04, = 0.005, = 0.4

Прогресс: 244/256

  = 0.2, = 0.04, = 0.005, = 0.5

Прогресс: 245/256

  = 0.2, = 0.04, = 0.01, = 0.2

Прогресс: 246/256

  = 0.2, = 0.04, = 0.01, = 0.3

Прогресс: 247/256

  = 0.2, = 0.04, = 0.01, = 0.4

Прогресс: 248/256

  = 0.2, = 0.04, = 0.01, = 0.5

Прогресс: 249/256

```
  □ = 0.2, □ = 0.04, □ = 0.015, □ = 0.2

Прогресс: 250/256
  □ = 0.2, □ = 0.04, □ = 0.015, □ = 0.3

Прогресс: 251/256
  □ = 0.2, □ = 0.04, □ = 0.015, □ = 0.4

Прогресс: 252/256
  □ = 0.2, □ = 0.04, □ = 0.015, □ = 0.5

Прогресс: 253/256
  □ = 0.2, □ = 0.04, □ = 0.02, □ = 0.2

Прогресс: 254/256
  □ = 0.2, □ = 0.04, □ = 0.02, □ = 0.3

Прогресс: 255/256
  □ = 0.2, □ = 0.04, □ = 0.02, □ = 0.4

Прогресс: 256/256
  □ = 0.2, □ = 0.04, □ = 0.02, □ = 0.5
```

6.19 10.5 Сводная таблица результатов

```
results_df = DataFrame(all_results)
println("\n" * "="^60)
println("СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ")
```

```
println("="^60)
println(first(results_df, 10))  # показываем первые 10 строк
```

```
============================================================
СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ
============================================================
10×12 DataFrame
 Row │ □      u0        □      □      mean_prey  tspan      prey_amplitude □      mean_pre
     │ Float64  Array…     Float64  Float64  Float64   Tuple…     Float64      Float64  Fl
─────┼──────────────────────────────────────────────────────────────────────────────────────
   1 │   0.05  [40.0, 9.0]   0.005    0.01   39.7947  (0.0, 200.0)      13.0597    0.2
   2 │   0.05  [40.0, 9.0]   0.005    0.01   59.0119  (0.0, 200.0)      28.0659    0.3
   3 │   0.05  [40.0, 9.0]   0.005    0.01   80.7191  (0.0, 200.0)      53.5718    0.4
   4 │   0.05  [40.0, 9.0]   0.005    0.01  100.421   (0.0, 200.0)      83.8135    0.5
   5 │   0.05  [40.0, 9.0]   0.01     0.01   18.6958  (0.0, 200.0)      17.3045    0.2
   6 │   0.05  [40.0, 9.0]   0.01     0.01   29.2117  (0.0, 200.0)      12.1603    0.3
   7 │   0.05  [40.0, 9.0]   0.01     0.01   39.392   (0.0, 200.0)       9.2436    0.4
   8 │   0.05  [40.0, 9.0]   0.01     0.01   50.0495  (0.0, 200.0)      14.9469    0.5
   9 │   0.05  [40.0, 9.0]   0.015    0.01   13.0688  (0.0, 200.0)      19.401     0.2
  10 │   0.05  [40.0, 9.0]   0.015    0.01   19.9553  (0.0, 200.0)      16.8798    0.3

  ============================================================ 11.
ВИЗУАЛИЗАЦИЯ ПАРАМЕТРИЧЕСКОГО СКАНИРОВАНИЯ ==============================
```

6.20 11.1 Зависимость средних значений от параметров

Исследуем, как параметры влияют на средние численности популяций.

График: зависимость средних значений от α

```
p_alpha = scatter(results_df.□, [results_df.mean_prey results_df.mean_predator],
    label=[L"\bar{x}" L"\bar{y}"],
    xlabel=L"\alpha (скорость размножения жертв)",
    ylabel="Средняя численность",
    title="Зависимость средних значений от □",
    color=[:green :red],
    markersize=4,
    alpha=0.7,
    grid=true)

savefig(p_alpha, plotsdir(script_name, "param_scan_alpha.png"))
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-
1--study--simulationmod/labs/lab02/project/plots/param_scan_alpha.png"
```

График: зависимость средних значений от β

```
p_beta = scatter(results_df.□, [results_df.mean_prey results_df.mean_predator],
    label=[L"\bar{x}" L"\bar{y}"],
    xlabel=L"\beta (скорость поедания)",
    ylabel="Средняя численность",
    title="Зависимость средних значений от □",
    color=[:green :red],
    markersize=4,
    alpha=0.7,
    grid=true)

savefig(p_beta, plotsdir(script_name, "param_scan_beta.png"))
```

”/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab02/project/plots/param_scan_beta.png”

6.21 11.2 Зависимость амплитуды колебаний от параметров

```
p_amplitude = scatter(results_df.□, [results_df.prey_amplitude results_df.predator_am
    label=[L”A_x” L”A_y”],
    xlabel=L”\alpha”,
    ylabel=”Амплитуда колебаний”,
    title=”Зависимость амплитуды от □”,
    color=[:green :red],
    markersize=4,
    alpha=0.7,
    grid=true)

savefig(p_amplitude, plotsdir(script_name, ”param_scan_amplitude.png”))
```

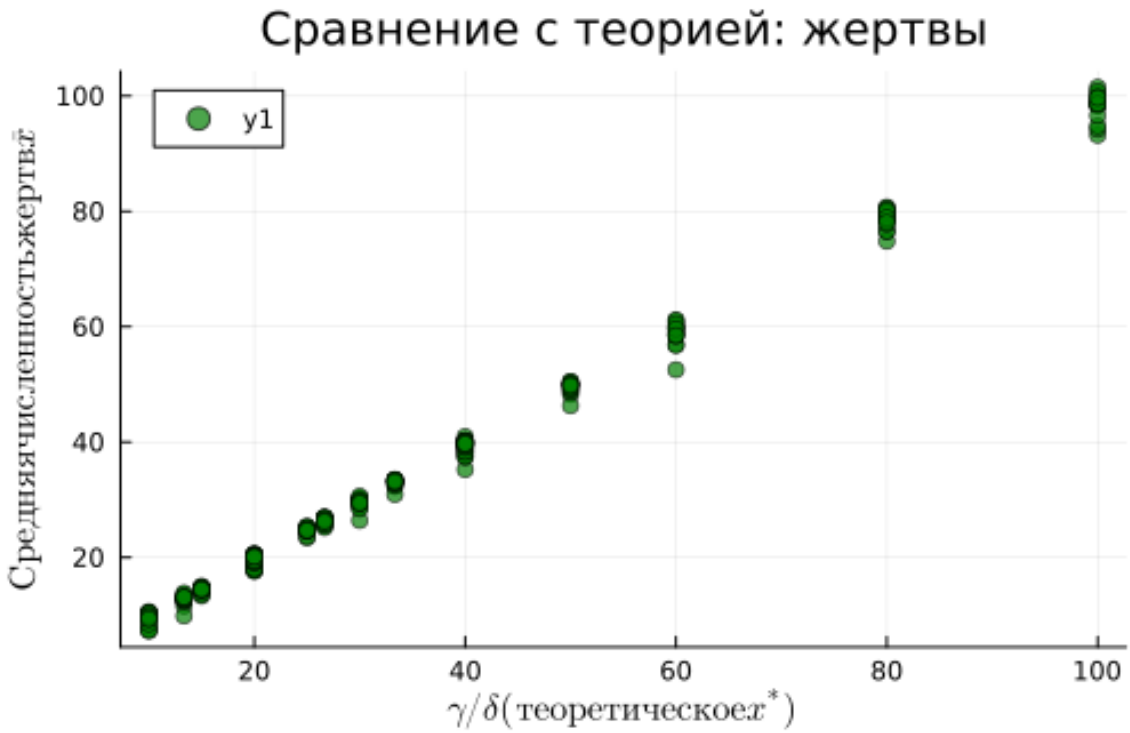
”/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab02/project/plots/param_scan_amplitude.png”

6.22 11.3 Сравнение с теорией: стационарные точки

Теоретические стационарные точки:

```
x^* = \frac{\gamma}{\delta}, \quad y^* = \frac{\alpha}{\beta}
```

```
p_theory_x = scatter(results_df.□ ./ results_df.□, results_df.mean_prey,
    xlabel=L"\gamma/\delta (теоретическое x^*)",
    ylabel=L"Средняя численность жертв \bar{x}",
    title="Сравнение с теорией: жертвы",
    color=:green,
    markersize=4,
    alpha=0.7,
    grid=true)
```



Линия y = x для сравнения

```
min_val = min(minimum(results_df.□ ./ results_df.□), minimum(results_df.mean_prey))
max_val = max(maximum(results_df.□ ./ results_df.□), maximum(results_df.mean_prey))
plot!(p_theory_x, [min_val, max_val], [min_val, max_val],
    color=:red, linestyle=:dash, label="Теория: x* = □/□")
```

```
savefig(p_theory_x, plotsdir(script_name, "param_scan_theory_x.png"))

p_theory_y = scatter(results_df.□ ./ results_df.□, results_df.mean_predator,
    xlabel=L"\alpha/\beta (теоретическое y^*)",
    ylabel=L"Средняя численность хищников \bar{y}",
    title="Сравнение с теорией: хищники",
    color=:red,
    markersize=4,
    alpha=0.7,
    grid=true)

plot!(p_theory_y, [min_val, max_val], [min_val, max_val],
    color=:green, linestyle=:dash, label="Теория: y* = □/□")

savefig(p_theory_y, plotsdir(script_name, "param_scan_theory_y.png"))
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-
1--study--simulationmod/labs/lab02/project/plots/param_scan_theory_y.png"
```

6.23 11.4 Тепловая карта: влияние α и β на амплитуду

Создадим матрицу для тепловой карты

```
□_vals = sort(unique(results_df.□))
□_vals = sort(unique(results_df.□))
amplitude_matrix = zeros(length(□_vals), length(□_vals))

for i in 1:length(□_vals)
    for j in 1:length(□_vals)
```

```
        row = findfirst((results_df.β .== β_vals[i]) .&
                        (results_df.α .== α_vals[j]))
        if !isnothing(row)
            amplitude_matrix[i, j] = results_df.prey_amplitude[row]
        end
    end
end

p_heatmap = heatmap(β_vals, α_vals, amplitude_matrix,
    xlabel=L"\beta",
    ylabel=L"\alpha",
    title="Тепловая карта: амплитуда жертв",
    color=:viridis,
    colorbar_title="Амплитуда")

savefig(p_heatmap, plotsdir(script_name, "param_scan_heatmap.png"))
```

"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab02/project/plots/param_scan_heatmap.png"

========== 12. АНАЛИЗ РЕЗУЛЬТАТОВ ПАРАМЕТРИЧЕСКОГО СКАНИРОВАНИЯ ==========

```
println("\n" * "="^60)
println("АНАЛИЗ РЕЗУЛЬТАТОВ ПАРАМЕТРИЧЕСКОГО СКАНИРОВАНИЯ")
println("="^60)

println("\nСтатистика по всем экспериментам:")
println("  Средняя численность жертв: от $(round(minimum(results_df.mean_prey), digit
```

```
        "до $(round(maximum(results_df.mean_prey), digits=2))")
println("  Средняя численность хищников: от $(round(minimum(results_df.mean_predator)
        "до $(round(maximum(results_df.mean_predator), digits=2))")
println("  Амплитуда жертв: от $(round(minimum(results_df.prey_amplitude), digits=2))
        "до $(round(maximum(results_df.prey_amplitude), digits=2))")
println("  Амплитуда хищников: от $(round(minimum(results_df.predator_amplitude), dig
        "до $(round(maximum(results_df.predator_amplitude), digits=2))")
```

```
============================================================
АНАЛИЗ РЕЗУЛЬТАТОВ ПАРАМЕТРИЧЕСКОГО СКАНИРОВАНИЯ
============================================================

Статистика по всем экспериментам:
  Средняя численность жертв: от 7.28 до 101.49
  Средняя численность хищников: от 1.06 до 21.44
  Амплитуда жертв: от 2.18 до 126.61
  Амплитуда хищников: от 1.06 до 42.64
```

Находим эксперименты с максимальной и минимальной амплитудой

```
max_amp_idx = argmax(results_df.prey_amplitude)
min_amp_idx = argmin(results_df.prey_amplitude)

println("\n□ Эксперимент с МАКСИМАЛЬНОЙ амплитудой жертв:")
println("  □ = $(results_df.□[max_amp_idx]), □ = $(results_df.□[max_amp_idx]), " *
        "□ = $(results_df.□[max_amp_idx]), □ = $(results_df.□[max_amp_idx])")
println("  Амплитуда жертв: $(round(results_df.prey_amplitude[max_amp_idx], digits=2)
```

```
println("\n□ Эксперимент с МИНИМАЛЬНОЙ амплитудой жертв:")
println("  □ = $(results_df.□[min_amp_idx]), □ = $(results_df.□[min_amp_idx]), " *
        "□ = $(results_df.□[min_amp_idx]), □ = $(results_df.□[min_amp_idx])")
println("  Амплитуда жертв: $(round(results_df.prey_amplitude[min_amp_idx], digits=2)
```

```
□ Эксперимент с МАКСИМАЛЬНОЙ амплитудой жертв:
  □ = 0.05, □ = 0.04, □ = 0.005, □ = 0.5
  Амплитуда жертв: 126.61

□ Эксперимент с МИНИМАЛЬНОЙ амплитудой жертв:
  □ = 0.1, □ = 0.01, □ = 0.01, □ = 0.4
  Амплитуда жертв: 2.18

  ============================================================
```

13. СОХРАНЕНИЕ РЕЗУЛЬТАТОВ ==========

```
println("\n" * "="^60)
println("СОХРАНЕНИЕ РЕЗУЛЬТАТОВ")
println("="^60)
```

```
============================================================
СОХРАНЕНИЕ РЕЗУЛЬТАТОВ
============================================================
```

Сохранение графиков базового эксперимента

```julia
savefig(plt1, plotsdir(script_name, "lv_dynamics.png"))
savefig(plt2, plotsdir(script_name, "lv_phase_portrait.png"))
savefig(plt3, plotsdir(script_name, "lv_derivatives.png"))
savefig(plt4, plotsdir(script_name, "lv_relative_changes.png"))
savefig(plt5, plotsdir(script_name, "lv_spectrum.png"))
savefig(plt6, plotsdir(script_name, "lv_panel.png"))

println("  □ Графики базового эксперимента сохранены в: plots/$(script_name)/")
```

```
  □ Графики базового эксперимента сохранены в: plots//

┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xsca
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xsca
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: No strict ticks found
└ @ PlotUtils ~/.julia/packages/PlotUtils/HX80C/src/ticks.jl:194
┌ Warning: Invalid negative or zero value 0.0 found at series index 1 for log10 based xsca
└ @ Plots ~/.julia/packages/Plots/GIume/src/utils.jl:105
```

Сохранение данных базового эксперимента

```
@save datadir(script_name, "lv_results.jld2") df_lv p_lv x_star y_star
println("  □ Данные базового эксперимента сохранены в: data/$(script_name)/lv_results
```

```
  □ Данные базового эксперимента сохранены в: data//lv_results.jld2
```

Сохранение результатов параметрического сканирования

```
CSV.write(datadir(script_name, "parameter_scan_results.csv"), results_df)
println("  □ Таблица результатов сканирования: data/$(script_name)/parameter_scan_res
```

```
  □ Таблица результатов сканирования: data//parameter_scan_results.csv
```

Сохранение всех данных параметрического сканирования

```
data_to_save = Dict(
    "all_params" => all_params,
    "all_results" => all_results,
    "results_df" => results_df,
    "param_grid" => param_grid
)
save(datadir(script_name, "lv_parameter_scan_complete.jld2"), data_to_save)
println("  □ Полные данные сканирования: data/$(script_name)/lv_parameter_scan_comple
```

```
  □ Полные данные сканирования: data//lv_parameter_scan_complete.jld2
```

========== 14. ЗАВЕРШЕНИЕ РАБОТЫ ==========

```
println("\n" * "="^60)
println("□ МОДЕЛИРОВАНИЕ ЗАВЕРШЕНО УСПЕШНО!")
println("="^60)
```

```
println("\nВсего выполнено экспериментов: $(length(all_params))")
println("  - Базовый эксперимент: 1")
println("  - Параметрическое сканирование: $(length(all_params))")
println("\nРезультаты сохранены в:")
println("  □ plots/$(script_name)/ - все графики")
println("  □ data/$(script_name)/ - все данные")
```

```
============================================================
□ МОДЕЛИРОВАНИЕ ЗАВЕРШЕНО УСПЕШНО!
============================================================

Всего выполнено экспериментов: 256
  - Базовый эксперимент: 1
  - Параметрическое сканирование: 256

Результаты сохранены в:
  □ plots// - все графики
  □ data// - все данные
```

7 Выводы

Выполнив эту лабораторную работу, я получила практическое знание о моделях SIR и Лотки-Вольтерры с использованием языка Джулии.